EYEFLOW: WEB-BASED EYE TRACKINGMOUSE FORHANDS-FREE COMPUTER INTERACTION

(PROJECT PHASE - II)
submitted in partial fulfillment of the requirements
for the award of the degree in

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING

By

D. PARTHIBAN (Reg. No. 221061101200)
D. MOHAMMED RAFEEK (Reg. No. 221061101176)
MOHAMMED FAHIJ (Reg. No. 221061101174)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

APRIL 2026

Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY
University with Graded Autonomy Status
(An ISO 21001 : 2018 Certified Institution)
Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this Project Report (Project Phase-II) is the Bonafide work of **D.PARTHIBAN** Reg. **No. 221061101200**, **D.MOHAMMED RAFEEK** Reg. **No. 221061101176, Y.MOHAMMED FAHJI** Reg. **No. 221061101174**, who carried out the project entitled **"EYEFLOW: WEB-BASED EYE TRACKING MOUSE FOR HAND-FREE COMPUTER INTERACTION"** under our supervision from December 2025 to April 2026.

Internal Guide **Mr. A. Jegadeeswari** **Assistant Professor** Department of CSE Dr. M.G.R Educational And Research Institute Deemed to be University	**Project Coordinator** **Dr.T.kumanan Professor** **Dr.M.Nisha Assistant Professor** Department of CSE Dr. M.G.R Educational And Research Institute Deemed to be University	**Department Head** **Dr. S. Geetha** **Professor and HOD** Department of CSE Dr. M.G.R. Educational and Research Institute Deemed to be University

Submitted for Viva Voce Examination held on _____

Internal Examiner **External Examiner**

Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY
University with Graded Autonomy Status
(An ISO 21001 : 2018 Certified Institution)
Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.

DECLARATION

We, **D.PARTHIBAN (221061101200), D.MOAHMMED RAFEEK (221061101200), D.MOHAMEED FAHIJ (221061101176),** here by declare that the Project entitled **"EYEFLOW: WEB-BASED EYE TRACKING MOUSE FOR HANDS-FREE COMPUTER INTERACTION"** is done by us under the guidance of **MS.A.JEGADEESWARI** and is submitted in partial fulfilment of the requirements for the award of the degree in **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGNEERING**

SIGNATURE OF THE CANDIDATES

DATE: _____

PLACE: _____ 1.

2.

3.

ACKNOWLEDGEMENT

We would first like to express our sincere gratitude to our beloved Respected Chancellor, **Thiru. Dr. A. C. SHANMUGAM, B.A., B.L.,** our Honourable President, **Er. A. C. S. ARUNKUMAR, B.Tech., M.B.A**., and our Secretary, **Thiru A. RAVIKUMAR,** for all the encouragement and support extended to us during the tenure of this project and also our years of studies in their Wonderful University.

We extend our heartfelt thanks to our Vice Chancellor, **Prof. Dr GEETHALAKSHMI,** for her invaluable support in facilitating our project.

We are also deeply grateful to our Head of the Department, **Prof. Dr. S. GEETHA,** whose continuous involvement and encouragement from the beginning to the completion of this project.

Our sincere thanks go to our Project Coordinators**, Dr. T. Kumanan** and **Dr. M.Nisha,** as well as our Project Supervisor, **Ms. A. Jegadeeswari,** for their consistent guidance, insightful suggestions, and unwavering encouragement, which were instrumental in the success of this work.

We also extend our gratitude to all the teaching and non-teaching staff of the Department of Computer Science and Engineering for their constant support and the encouragement they provided throughout our academic journey and during the development of this project.

ABSTRACT

Traditional human–computer interaction depends on physical devices such as keyboards and mice, which require fine motor control and remain inaccessible to many users with motor impairments. With nearly 1.3 billion people worldwide (16%) affected, the need for affordable assistive solutions is highly significant.

This work presents EyeFlow, a software-only, web-enabled hands-free interaction system that operates entirely using a standard laptop webcam without additional hardware. Built on the Flask framework, it combines gaze-based cursor control, right-eye wink clicking, nose-tilt scrolling, and continuous voice typing into a single accessible interface.

The system uses a head-compensated gaze estimation model based on the relative displacement between iris and nose-tip landmarks extracted using MediaPipe Face Mesh, which provides 468 three-dimensional landmarks at 30 FPS. This relative positioning decouples eye movement from head translation, ensuring stable and posture-independent cursor control.

Click events are detected using the Eye Aspect Ratio (EAR) algorithm through asymmetric right-eye closure while suppressing natural blinks. Vertical head movement is mapped to scrolling using asymmetric nose-tilt estimation, and a background-threaded Google Speech Recognition module enables continuous voice typing.

Experimental evaluation with ten participants achieved a 3.8% mean cursor placement error, 95.2% wink detection accuracy, and 62 ms end-to-end latency. A 33.3% improvement in cursor accuracy over 48 hours demonstrates strong learnability and usability.

The system runs on standard laptop hardware with 23% CPU utilization, requires internet only for voice typing, and offers nearly 95% cost reduction compared with commercial systems such as Tobii Eye Tracker 5.

Keywords: *Eye tracking, MediaPipe Face Mesh, EAR blink detection, gaze estimation, assistive technology, hands-free interaction, head-compensated cursor control, Flask, webcam-based HCI.*

TABLE OF CONTENTS

CHAPTER NO.	TITLE		PAGE NO.
	LIST OF TABLES		x
	LIST OF FIGURES		xi
	LIST OF ABBREVIATIONS		xii
	ABSTRACT		v
1	**INTRODUCTION**		1
	1.1	Objective of the Project	1
	1.2	Problem Statement	2
	1.3	Motivation	3
	1.4	Scope of the Project	4
	1.5	Ethical and Social Relevance	4
	1.6	Project Introduction	5
2	**LITERATURE SURVEY**		7
3	**SYSTEM SPECIFICATION**		12
	3.1	Existing System	14
	3.1.1	Disadvantages of the Existing System	16
	3.2	Proposed System	17
	3.3	Functional Requirements	19
	3.4	Non-Functional Requirements	21
	3.5	Hardware Requirements	21
	3.6	Software Requirements	21
	3.7	Feasibility Analysis	22

CHAPTER NO.		TITLE	PAGE NO.
	3.8	Maintainability	22
4		**SYSTEM DESIGN**	23
	4.1	Overall System Architecture	18
	4.2	Six-Layer Architecture Diagram	19
	4.3	Processing Pipeline	20
	4.4	UML Diagrams	22
	4.4.1	Use Case Diagram	22
	4.4.2	Sequence Diagram	23
	4.4.3	Activity Diagram	24
	4.5	Data Flow Diagram	25
	4.6	Module Interaction Design	22
	4.7	Database and JSON Storage Structure	25
	4.8	Standards Used	25
5		**IMPLEMENTATION AND METHODOLOGY**	26
	5.1	Webcam Frame Capture	26
	5.2	Face Mesh Landmark Extraction	27
	5.3	Head-Compensated Gaze Estimation	28
	5.4	Mathematical Formulation	29
	5.5	Right-Eye Wink Detection	30
	5.4.1	EAR Algorithm	31
	5.4.2	Wink State Machine	31
	5.5	Nose-Tilt Scroll Control	32
	5.6	Five-Point Automatic Calibration	33

CHAPTER NO.		TITLE	PAGE NO.
	5.7	Voice Typing Module	34
	5.8	Pseudocode Algorithms	35
6		**TESTING AND RESULTS**	36
	6.1	Test Plan	36
	6.2	Testing Environment	37
	6.3	User Study Design	37
	6.4	Cursor Accuracy Results	38
	6.5	Wink Detection Results	39
	6.6	System Performance Results	41
	6.7	Learning Curve Analysis	42
	6.8	Head Movement Robustness	43
	6.9	Comprehensive Comparison with Existing Systems	44
7		**CONCLUSION AND FUTURE ENHANCEMENT**	46
	7.1	Conclusion	46
	7.2	Project Impact and Accessibility Benefits	47
	7.3	Limitations	48
	7.4	Future Scope	49
		REFERENCES	51
		APPENDIX	55

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
Table 2.1	Capability Comparison of Existing Systems with EyeFlow	11
Table 2.2	Summary of Key Literature	12
Table 3.1	Hardware Requirements for EyeFlow	14
Table 3.2	Software Requirements for EyeFlow	15
Table 4.1	Module Specification Table	22
Table 4.2	Data Storage Structure	25
Table 5.1	Wink Detection State Machine	31
Table 5.2	Voice Typing Recognition Parameters	34
Table 6.1	Cursor Positioning Error by Screen Region	38
Table 6.2	Wink Detection Performance Summary	39
Table 6.3	Wink Detection by User Group	40
Table 6.4	System Resource Usage and Latency Metrics	41
Table 6.5	Cursor Error Across Three Sessions (Learning Curve)	42
Table 6.6	Head Displacement Robustness — EyeFlow vs Absolute Mapping	43
Table 6.7	EyeFlow vs Tobii Eye Tracker 5 vs WebGazer	44

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
Fig 4.1	EyeFlow Six-Layer Architecture	19
Fig 4.2	EyeFlow Frame Processing Pipeline	21
Fig 4.3	Use Case Diagram of EyeFlow	22
Fig 4.4	Sequence Diagram — Gaze Cursor Control Flow	23
Fig 4.5	Activity Diagram — EyeFlow Tracking Session	24
Fig 4.6	Level-0 Data Flow Diagram of EyeFlow	25
Fig 4.7	Level-1 Data Flow Diagram of EyeFlow	25
Fig 6.1	Bar Chart: EyeFlow vs Tobii 5 vs WebGazer Feature Comparison	45
Fig 6.2	Learning Curve: Cursor Error Across Three Sessions	45
Fig 6.3	Fatigue Analysis: 30-Minute Extended Session	46

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
BGR	Blue Green Red (colour format used by OpenCV)
CPU	Central Processing Unit
CCTV	Closed Circuit Television
DXA	Document Extended Attribute (measurement unit)
EAR	Eye Aspect Ratio
EMA	Exponential Moving Average
FPS	Frames Per Second
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HCI	Human-Computer Interaction
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IoT	Internet of Things
IR	Infrared
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
NAS	Neural Architecture Search
NMS	Non-Maximum Suppression
OS	Operating System
PPE	Personal Protective Equipment
REST	Representational State Transfer
RGB	Red Green Blue
SDK	Software Development Kit
SQL	Structured Query Language
STT	Speech-To-Text
UI	User Interface
WCAG	Web Content Accessibility Guidelines
WPM	Words Per Minute

CHAPTER 1

INTRODUCTION

1.1 Objective of the Project

The primary objective of this project is to design, develop, and evaluate a software-only, hands-free computer interaction system called EyeFlow that allows individuals with motor-related disabilities to control a computer cursor, perform mouse clicks, scroll through documents, and type text entirely through eye movement, facial gestures, and voice commands — using nothing more than a standard laptop webcam. The system is motivated by the global need for affordable and accessible assistive technology and aims to eliminate the hardware cost barrier that prevents millions of people from accessing such tools.

Specifically, the project targets the following measurable objectives. First, to implement a head-compensated gaze estimation model capable of mapping iris displacement relative to the nose tip onto screen coordinates with a mean cursor placement error of less than five percent of the screen diagonal. Second, to develop a wink-based click detection mechanism using the Eye Aspect Ratio (EAR) algorithm that achieves a true-positive detection rate of at least ninety percent while maintaining a false-positive rate below five percent. Third, to build a nose-tilt scroll controller that uses asymmetric gain factors to equalize perceived scroll speed in both upward and downward directions. Fourth, to integrate continuous voice typing through Google Speech Recognition in a background processing thread that does not degrade visual tracking performance. Fifth, to validate the complete system through a structured user study involving a minimum of ten participants across at least three sessions each.

Beyond the technical targets, the project also aims to demonstrate that a well-engineered software solution can realistically compete with entry-level commercial eye-tracking hardware — offering approximately ninety-five percent cost savings while remaining practically usable in everyday computing scenarios.

1.2 Problem Statement

Human-computer interaction has historically assumed that users possess reliable fine motor control over their hands and fingers. This assumption is embedded deeply into the design of conventional input devices, from the standard QWERTY

keyboard to the optical mouse. However, this assumption is fundamentally flawed when considered against the global disability landscape. According to the World Health Organization's 2022 Global Report on Assistive Technology, approximately 1.3 billion people — representing 16 percent of the world's population — live with some form of significant disability, and a substantial proportion of these individuals have motor impairments that prevent them from reliably operating standard input devices.

The existing market for assistive eye-tracking technology does offer some solutions, but these come at a significant economic cost. The Tobii Eye Tracker 5, considered an entry-level consumer product in the category, retails at approximately USD 229. Professional and research-grade systems such as the EyeTech TM5 cost nearly USD 4,000, while clinical-grade devices can exceed USD 16,000. These costs place the technology entirely out of reach for the majority of users in low- and middle-income countries, as well as for individuals in developed economies who lack insurance coverage for assistive devices. The WHO report further indicates that only one in ten people in need of assistive technology actually acquires it, citing cost and access as the primary barriers.

Beyond cost, commercial eye-tracking solutions impose additional barriers. They typically require proprietary hardware — including infrared LED arrays, depth sensors, and vendor-specific drivers — that must be professionally calibrated and configured. This dependency on specialised hardware limits usability to controlled environments and prevents daily home computing use. Furthermore, most commercial systems provide only cursor control, without integrating scrolling, clicking, and text input into a single unified interface.

Webcam-based approaches have been explored in research, most notably through the WebGazer library developed by Papoutsaki et al., which demonstrated that implicit gaze calibration through browser click events was feasible. However, WebGazer is limited to estimating gaze within a browser tab and provides no path to system-wide cursor control, nor does it offer click, scroll, or text input channels. No existing open-source, webcam-based solution simultaneously provides head-compensated gaze cursor control, differentiating wink from blink for click triggering, nose-tilt scrolling, and integrated voice typing in a single deployable application. This gap defines the problem that EyeFlow is designed to address.

1.3 Motivation

The motivation behind EyeFlow is rooted in both social responsibility and technological opportunity. From a social perspective, the inability to afford or access assistive technology represents a profound equity issue. Individuals who cannot use standard input devices are effectively excluded from the digital economy — unable to work remotely, access online education, communicate through social media, or interact with government services. In a world increasingly mediated through digital interfaces, this exclusion is not merely an inconvenience but a significant barrier to participation in modern society.

From a technological perspective, the convergence of several mature and freely available tools creates an unprecedented opportunity. MediaPipe Face Mesh, published by Google and released as an open-source library, enables real-time detection of 468 three-dimensional facial landmarks at interactive frame rates on a standard laptop CPU, without requiring any specialised optical hardware. Python's ecosystem of automation libraries — including PyAutoGUI for system-level mouse and keyboard control, OpenCV for video capture and image processing, and Flask for lightweight web server deployment — provides all the building blocks needed for a comprehensive assistive interface. The Google Speech Recognition API, available at no cost for standard usage volumes, provides cloud-based speech-to-text transcription that rivals dedicated hardware solutions.

The combination of these tools means that a comprehensive, multi-modal assistive interface can be built and distributed at zero incremental hardware cost to the end user. Anyone with a laptop manufactured in the last decade — whether in Chennai, rural Kenya, or rural Nebraska — can potentially access the same quality of hands-free computer control. This democratisation of assistive technology is the central motivation driving the development of EyeFlow.

Additionally, the project is motivated by the academic opportunity to contribute a structured, quantitatively validated study of webcam-based multi-modal interaction to the research community. By conducting a controlled user study across three sessions with ten participants, documenting learning curve effects, fatigue metrics, and comparative performance against commercial baselines, EyeFlow contributes data that is rarely available in the assistive technology literature for software-only systems.

1.4 Scope of the Project

The scope of EyeFlow encompasses the design, implementation, testing, and validation of a complete hands-free computer control system delivered as a web application running on a standard Flask server. The system is scoped to operate on Windows-based laptop computers with a built-in 720p webcam and an internet connection (required only for the voice typing module). The target operating environment is a standard indoor office or home environment with typical artificial or natural lighting.

Within this scope, the project covers four primary interaction modalities. The first modality is gaze-based cursor control, which encompasses the full processing pipeline from webcam frame capture, through MediaPipe landmark extraction, head-compensated iris-nose displacement computation, calibration-range normalisation, exponential moving average smoothing, and system-level cursor positioning using PyAutoGUI. The second modality is wink-based click triggering, which covers EAR computation, bilateral blink rejection through inter-eye EAR asymmetry, a four-state wink detection state machine, adaptive EAR baseline calibration, and click event dispatch with a post-click cursor freeze to prevent iris-movement artifacts.

The third modality is nose-tilt scroll control, which covers nose-tip offset computation from the calibration reference, a dead-zone filter to suppress involuntary micro-movements, asymmetric gain factors for up and down scrolling, and rate-limited scroll event dispatch. The fourth modality is voice typing, which covers microphone input capture through Python's SpeechRecognition library, asynchronous audio segmentation and recognition using the Google Speech-to-Text API, and clipboard-paste text insertion into the active application.

The system includes a five-point automatic calibration procedure that runs once per user and stores per-user calibration data persistently using Python pickle serialisation, eliminating the need for repeat calibration on subsequent sessions. Multi-user profile support, bcrypt-based user authentication, and session management are included in the web application layer. The scope explicitly excludes custom hardware development, infrared-based tracking, real-time model training, and multi-person simultaneous tracking.

1.5 Ethical and Social Relevance

EyeFlow carries significant ethical and social dimensions. The project directly addresses the digital inclusion of people with disabilities, a group that is frequently marginalised in technology development. By prioritising zero hardware cost, cross-platform web deployment, and local video processing without external data transmission, EyeFlow embodies the principles of inclusive design and privacy-respecting technology.

Privacy is a particularly important consideration in a system that continuously processes video of a user's face. EyeFlow is deliberately designed so that all facial landmark analysis occurs entirely on the local device — no video frames, face images, or biometric data are transmitted to external servers. Only the voice typing module communicates with an external API, and this communication is limited to anonymised audio snippets. This local-processing architecture stands in contrast to many commercial and cloud-based assistive systems, which upload biometric data to vendor servers for processing.

The system's design also reflects awareness of the cognitive and physical fatigue that accompanies extended eye-tracking use — a phenomenon documented in the human-computer interaction literature. EyeFlow includes a dead zone in the scroll controller to prevent accidental input, an adaptive EAR baseline that compensates for eyelid droop caused by fatigue, and a post-click cursor freeze to prevent the iris movement that accompanies winking from displacing the cursor. The experimental evaluation includes a 30-minute fatigue session that quantifies the degradation in cursor accuracy and subjective fatigue scores, enabling the project to make evidence-based recommendations about usage duration.

From a societal perspective, a system like EyeFlow could meaningfully improve the daily quality of life of individuals with conditions such as amyotrophic lateral sclerosis (ALS), cerebral palsy, muscular dystrophy, spinal cord injuries, and severe repetitive strain injuries. It could enable remote work, access to online education, social communication, and independent use of government services — opportunities that are currently denied to many individuals due to the combination of disability and economic constraint.

1.6 Project Introduction

EyeFlow is a web-enabled, multi-modal assistive interface designed to give individuals with motor impairments complete hands-free control of a desktop computer. The system is built around Google's MediaPipe Face Mesh library, which enables sub-pixel detection of iris landmarks from a standard 720p webcam feed at 30 frames per second. From these landmarks, EyeFlow derives four independent control channels: gaze-based cursor movement, wink-triggered mouse clicking, nose-tilt-driven page scrolling, and speech-driven text typing.

The architecture of EyeFlow is organised into six horizontal layers. The user interface layer consists of three HTML5 pages — a landing page, a user dashboard, and an active tracking view — served by a Flask 3.0 web application. Below the application server layer sits the business logic layer, which contains five independent modules: the eye-tracking engine, the EAR-based wink detector, the nose-tilt scroll controller, the calibration manager, and the voice typing engine. These modules communicate exclusively through a shared state object, ensuring clean separation of concerns. The computer vision layer processes each webcam frame through OpenCV capture and MediaPipe inference to produce 468 normalised landmark coordinates. The system integration layer uses PyAutoGUI to translate computed gaze coordinates and gesture events into operating system-level mouse and keyboard actions. The data layer stores user credentials and calibration parameters in JSON and pickle files respectively.

A critical technical contribution of EyeFlow is the head-compensated gaze model. Conventional webcam trackers map the absolute iris position to screen coordinates, meaning that any head movement shifts the cursor even when the user's gaze direction has not changed. EyeFlow decouples head translation from gaze direction by computing the relative displacement of the iris with respect to the nose tip — a facial reference point that moves with the head but not with voluntary eye rotation. This simple but effective technique keeps the cursor stable under natural postural variation of up to 15 centimetres lateral head displacement, achieving a 5.29-fold improvement in cursor accuracy at that displacement compared to absolute iris mapping.

The system has been validated through a structured user study with ten university participants across three sessions. Results demonstrate a mean cursor error of 3.8 percent of the screen diagonal, a wink detection accuracy of 95.2 percent, and an

overall learning curve improvement of 33.3 percent over 48 hours. Running on a standard mid-range laptop, EyeFlow processes frames at 30 frames per second with 23 percent CPU usage and a total end-to-end cursor latency of 62 milliseconds — well within the 100-millisecond threshold established in the assistive HCI literature as the boundary for interactive cursor control.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview of Eye Tracking Technology

Eye tracking refers to the process of measuring and recording the direction of a person's gaze relative to a reference frame, typically a computer screen. The field has its roots in ophthalmological research from the late nineteenth century, but modern computational eye tracking emerged primarily in the context of reading research, cognitive science, and more recently, human-computer interaction. Hansen and Ji (2010) conducted a comprehensive survey of models for eyes and gaze, organising the literature into two major paradigms: feature-based methods and appearance-based methods. Feature-based methods extract geometric or photometric features from the eye region — such as corneal reflections from infrared LEDs, limbus contours, or iris boundaries — and use these features to compute gaze direction through geometric models calibrated per user. Appearance-based methods bypass explicit feature extraction and instead train machine learning models to regress gaze angles directly from pixel intensities in cropped eye-region images.

Both paradigms face fundamental challenges in natural, unconstrained conditions. Feature-based methods depend on controlled lighting — typically an infrared illuminator positioned at a fixed distance from the eye — to produce reliable corneal reflections. Without such lighting, the reflections are unstable and gaze estimation accuracy degrades significantly. Appearance-based methods require large training datasets that include the target user's head pose and illumination variation, and typically require GPU-based inference at runtime, limiting their deployment on consumer hardware. Duchowski (2017) added a task-centric dimension to this discussion, noting that assistive applications impose distinctively strict latency requirements: pointer control for interactive use demands round-trip latencies below 100 milliseconds, a threshold that most research systems cannot achieve outside laboratory settings.

2.2 MediaPipe Face Mesh and Iris Tracking

Lugaresi et al. (2019) published MediaPipe as a cross-platform toolkit for packaging GPU-accelerated perception pipelines into production systems. The MediaPipe Face Mesh model, described by Kartynnik et al. (2019), uses a single

monocular RGB frame to produce 468 three-dimensional landmark coordinates at interactive frame rates on mobile-class processors, without requiring an infrared projector or a depth camera. The model employs a two-stage pipeline: a face detection model first localises the face in the image, and an attention mesh model then regresses all 468 landmarks jointly with two special iris refinement points at landmark indices 468 and 469. The iris refinement model co-regresses face geometry and iris localisation, achieving sub-pixel iris centre estimation on a smartphone GPU.

The availability of these refined iris coordinates, combined with the model's ability to run on a standard laptop CPU at 30 frames per second, is the technical foundation that makes EyeFlow possible. Prior to MediaPipe Face Mesh, achieving real-time iris localisation from a standard webcam required either dedicated infrared hardware or computationally expensive CNN inference that could not run at interactive rates on consumer CPUs. MediaPipe Face Mesh removes both of these barriers simultaneously, opening the door to a new class of lightweight, software-only gaze estimation systems.

2.3 Eye Aspect Ratio for Blink and Wink Detection

Soukupova and Cech (2016) introduced the Eye Aspect Ratio as a simple, computationally efficient univariate signal for real-time blink detection. The EAR is defined as the ratio of the sum of two vertical eye opening distances to the horizontal eye span, computed from six periocular landmarks. During an eye blink, the vertical distances approach zero, causing the EAR to drop sharply and briefly before recovering. The authors demonstrated that a simple threshold on the EAR signal was sufficient for reliable blink detection using a typical laptop webcam, without requiring any machine learning at inference time.

However, the original EAR formulation computes a single average EAR across both eyes, which means it cannot distinguish between a unilateral wink — in which only one eye closes — and a bilateral blink — in which both eyes close simultaneously. This inability to distinguish wink from blink is a critical limitation for a click-triggering system: if every bilateral blink triggered a click, the false-action rate would make the system unusable, given that adults blink spontaneously 15 to 20 times per minute. EyeFlow extends the original EAR formulation by computing separate EAR values for the left and right eyes using independent sets of periocular landmarks, and applying an inter-eye asymmetry threshold that only triggers a click when the right eye closes while

the left eye remains open beyond a configurable margin. This differential wink detection approach reduces the false-positive rate to 2.1 percent while maintaining a true-positive rate of 95.2 percent.

2.4 WebGazer and Browser-Based Eye Tracking

Papoutsaki et al. (2016) developed WebGazer, a JavaScript library that estimates gaze within a browser using the device's front-facing webcam. WebGazer operates through implicit calibration — it observes where the user clicks during ordinary web browsing and uses these click events as noisy gaze labels to continuously update a regression model mapping webcam images to screen coordinates. This approach eliminates the need for a separate calibration phase and was demonstrated to achieve angular errors in the range of two to four degrees in realistic browsing scenarios.

While WebGazer represented a meaningful advance toward zero-setup, zero-cost eye tracking, its scope is fundamentally constrained. Gaze estimation is performed only within the active browser tab, providing no path to system-wide cursor control. The library provides no click, scroll, or text input channel, and the continuous implicit calibration approach makes accuracy difficult to predict or control. EyeFlow addresses all of these limitations by implementing system-wide cursor control through PyAutoGUI, integrating three additional interaction modalities beyond cursor movement, and using an explicit five-point calibration procedure that produces stable, per-user mapping parameters stored for reuse across sessions.

2.5 Commercial Eye Tracking Systems

The commercial eye tracking market is dominated by Tobii Technology, whose products span consumer, gaming, and research segments. The Tobii Eye Tracker 5 is the company's entry-level consumer product, retailing at approximately USD 229. It uses infrared LED illuminators and a dedicated near-infrared camera to achieve corneal-reflection-based gaze tracking at 33 frames per second with an angular accuracy of approximately 0.4 degrees. The system provides system-wide cursor control, dwell-based clicking, and integration with assistive software through the Tobii Experience application. However, it requires a proprietary USB device that must be mounted below the monitor, operates only on Windows, and provides no integrated scrolling channel or voice typing capability.

At the research and clinical end of the spectrum, systems such as the Tobii Pro Fusion, SR Research EyeLink 1000 Plus, and SMI RED500 offer sub-arc-minute angular accuracy and recording frequencies up to 2000 Hz, but at costs ranging from USD 15,000 to over USD 50,000. These systems are entirely unsuitable for home use by individuals with disabilities. The cost gulf between clinical-grade systems and consumer products, and between consumer products and software-only approaches, represents the core accessibility problem in the field — and the primary market gap that EyeFlow targets.

2.6 Multi-Modal Accessible Interaction

Ghosh, Dhanalakshmi, and Sethuraman (2023) conducted a comparative evaluation of hybrid gaze-and-speech interfaces against gaze-only interfaces for accessible computing tasks. Their results showed that integrating speech input alongside gaze-based cursor control reduced task error rates and delayed the onset of fatigue compared to gaze-only control. This finding directly informed the decision to include a voice typing channel in EyeFlow, recognising that multi-modal redundancy reduces the cognitive and physical load placed on any single interaction channel.

Majaranta and Bulling (2014) surveyed the practical barriers to real-world deployment of eye-tracking interfaces, identifying three central challenges: the Midas Touch problem, in which ambient gaze is mistakenly interpreted as intentional input; increasing cognitive load with prolonged use; and the absence of a natural visual rest position. EyeFlow addresses the Midas Touch problem through the wink-to-click mechanism — gaze is used exclusively for cursor positioning while intentional clicks require a deliberate unilateral wink — and through the nose-tilt scroll controller's dead zone, which prevents accidental scrolling during normal head posture. The fatigue issue is addressed through experimental measurement and the recommendation of a five-minute break after every twenty minutes of continuous use.

2.7 Identified Research Gap and Summary

A review of the existing literature reveals that while individual components of EyeFlow's functionality have been demonstrated in isolation, no single open-source, software-only system has previously integrated all of the following capabilities simultaneously: head-compensated gaze cursor control that decouples eye rotation from head translation; wink-based single-click detection with explicit bilateral blink

rejection; nose-tilt scroll control with asymmetric gain compensation; perpetual background voice typing; per-user calibration with persistent storage; and multi-user profile management — all deployed within a single webcam-dependent web application with local video processing. This combined gap defines EyeFlow's unique research contribution.

Table 2.1 Capability Comparison of Existing Systems with EyeFlow

Capability	**Hansen [3]**	**Krafka [7]**	**WebGazer**	**Tobii 5**	**EyeFlow**
Cost	Research	Research	Free	USD 229+	Free
Special Hardware	Yes (IR)	GPU	No	IR Camera	No
Real-time Cursor	No	No	Browser only	Systemwide	Systemwide
Click Detection	No	No	No	Yes	Yes (Wink)
Scroll Support	No	No	No	Yes	Yes (Nose)
Voice Typing	No	No	No	No	Yes
Head Compensation	Partial	Implicit	No	Yes (IR)	Yes
Multi-User Profiles	No	No	No	No	Yes
Blink vs Wink	No	No	No	Partial	Yes
Open Source	No	Partial	Yes	No	Yes
Local Privacy	N/A	No	Yes	Partial	Yes

2.8 Literature Summary Table

Table 2.2 Summary of Key Literature

S.No	Authors & Year	Title / System	Methodology	Key Finding	Limitation
1	Hansen & Ji (2010)	Eye Tracking Survey	Feature-based & appearance-based methods	Comprehensive taxonomy of gaze estimation methods	Unconstrained conditions unsolved
2	Lugaresi et al. (2019)	MediaPipe	Real-time face mesh pipeline	468 landmarks at 30fps on mobile CPU	Requires Google library
3	Kartynnik et al. (2019)	MediaPipe Face Mesh	Attention mesh architecture	Sub-pixel iris localisation	Single RGB frame only
4	Soukupova & Cech (2016)	EAR Blink Detection	Six-landmark eye aspect ratio	Real-time blink detection	Cannot distinguish wink from blink
5	Krafka et al. (2016)	iTracker / GazeCapture	CNN on eye regions	GPU-comparable webcam gaze accuracy	Requires GPU and large training set
6	Zhang et al. (2017)	Full-Face Gaze	Whole face appearance model	Better generalisation over eye-only	Still requires GPU inference
7	Papoutsaki et al. (2016)	WebGazer	Browser implicit calibration	Zero-setup gaze estimation	Browser-only, no click/scroll
8	Wood & Bulling (2014)	EyeTab	Model-based gaze on tablets	Per-user calibration outperforms model-only	Not a full assistive interface
9	Valliappan et al. (2020)	Smartphone Eye Tracking	Mobile front camera	Competitive angular accuracy	Acquisition task only, not control
10	Ghosh et al. (2023)	Gaze + Speech Interface	Hybrid multimodal HCI	Reduced fatigue vs gaze-only	Limited to specific task set
11	Majaranta & Bulling (2014)	HCI Eye Tracking Survey	Literature review	Midas Touch and fatigue documented	No software-only solution proposed

CHAPTER 3

SYSTEM SPECIFICATION

3.1 Existing System

The current landscape of computer accessibility for motor-impaired users is dominated by a combination of hardware-based eye trackers, specialised head-mouse devices, and voice-control software operating as independent applications. Hardware eye trackers such as the Tobii Eye Tracker 5 use infrared LED illuminators combined with dedicated near-infrared cameras to precisely locate the corneal reflection and pupil centre. The gaze vector is computed from the geometric relationship between the corneal reflection and the pupil centre, and is then mapped to screen coordinates using a calibration model derived from a nine-point calibration procedure.

These hardware-based systems achieve impressive angular accuracy — typically around 0.4 degrees for the Tobii Eye Tracker 5 — but they come with significant practical constraints. The infrared hardware must be physically mounted below the monitor and precisely aligned, requiring professional setup. The vendor-specific software stack runs exclusively on Windows and must be kept updated to remain compatible with the operating system. Users are locked into the vendor's software ecosystem and cannot modify or extend the functionality. The systems provide no integrated voice typing capability, meaning that users who wish to type must run a separate speech recognition application such as Dragon NaturallySpeaking alongside the eye tracker, with no native integration between the two modalities.

Browser-based solutions such as WebGazer offer an alternative that requires no special hardware, but are limited in scope. As described in the literature survey, WebGazer provides gaze estimation only within the browser window and has no pathway to system-wide cursor control or interaction outside of the browser. Voice-control systems such as Windows Voice Control and Apple Dictation provide text dictation capability but do not offer cursor control through gaze or facial gesture, meaning that users who cannot operate a mouse still cannot navigate the interface to position the text cursor before dictating.

In summary, the existing landscape consists of high-cost hardware solutions that provide good cursor control but lack integrated typing; low-cost browser-based solutions that provide limited gaze estimation but no system-wide cursor control; and

voice-only solutions that provide text input but no cursor navigation. No existing accessible solution integrates all four modalities — cursor control, clicking, scrolling, and voice typing — in a single software-only application.

3.1.1 Disadvantages of the Existing System

The existing systems suffer from several critical disadvantages that EyeFlow is designed to overcome. First, the prohibitive cost of commercial eye trackers places them out of reach for the majority of users globally. The cheapest commercially available dedicated eye tracker costs USD 229 — a sum that represents a significant proportion of monthly income in most developing countries and is not consistently covered by disability support schemes in developed economies. Second, the hardware dependency of infrared-based trackers means that users cannot access the technology unless they are physically present at a properly configured workstation, preventing mobile and flexible use.

Third, the absence of integrated multi-modal interaction in any single product means that motor-impaired users must configure, calibrate, and manage multiple independent assistive software applications simultaneously, creating a significant cognitive overhead. Fourth, commercial systems typically require repeat calibration at each session, adding time and effort to the process of beginning a computer interaction session. Fifth, no open-source, software-only alternative provides the combination of head-compensated gaze tracking, wink-based clicking with blink rejection, nose-tilt scrolling, and voice typing in a single unified interface. Sixth, privacy is a concern with cloud-connected commercial systems, which may upload biometric data to vendor servers without transparent disclosure of retention and usage policies.

3.2 Proposed System

EyeFlow proposes a fundamentally different approach to hands-free computer interaction. Rather than building around specialised hardware, EyeFlow is designed as a pure software system that leverages the webcam already present in virtually every modern laptop. The system is deployed as a Flask web application, making it accessible through any modern browser without requiring installation of any executable software on the user's machine beyond the Python runtime and its dependencies.

The proposed system achieves head-compensated gaze-based cursor control by computing the relative displacement of the iris landmark with respect to the nose-tip

landmark, both extracted from the MediaPipe Face Mesh output. This relative displacement is normalised against a per-user calibration range derived from a five-point automatic calibration procedure, then mapped to screen coordinates and smoothed using an exponential moving average filter. The result is a cursor that follows the user's gaze direction while remaining stable under natural head movement of up to 15 centimetres lateral displacement.

Click triggering is achieved through right-eye wink detection using the Eye Aspect Ratio computed independently for each eye. The system uses a four-state wink detection state machine that validates wink duration within a configurable window, enforces a cooldown between successive clicks, and freezes the cursor briefly after each click to prevent the iris movement associated with a wink from displacing the cursor. Natural bilateral blinks are rejected by monitoring the left eye EAR simultaneously with the right eye EAR and only triggering a click when the right eye closes while the left eye remains open above a threshold.

Scrolling is provided through nose-tilt detection, which measures the vertical offset of the nose tip from its calibration reference position and converts this offset into scroll events using asymmetric gain factors that equalise perceived scroll speed in both directions. Voice typing is implemented as a background processing thread that captures microphone audio, segments it into utterances, transcribes each utterance using the Google Speech-to-Text API, and pastes the transcribed text into the currently active application through a clipboard paste operation.

3.3 Functional Requirements

The functional requirements of EyeFlow define the specific behaviours that the system must exhibit to satisfy the needs of its target users. These requirements are organised by subsystem.

The authentication and user management subsystem must allow users to register with a username and password, with passwords stored as bcrypt hashes with a work factor of at least 12 rounds. Sessions must expire after 60 minutes of inactivity. The system must support multiple independent user profiles, each with separate calibration data and preferences.

The calibration subsystem must present a five-point automatic calibration procedure in which pulsing visual targets appear sequentially at screen positions

corresponding to the four corners and the centre. The system must collect at least eight valid landmark frames per calibration point, compute the iris-nose relative displacement range, pad the calibration boundaries by ten percent to accommodate users whose eye movement range slightly exceeds the calibrated range, and serialise the calibration parameters using Python pickle for persistent storage per user profile.

The gaze cursor control subsystem must process webcam frames at a sustained rate of at least 25 frames per second. It must extract iris and nose landmark coordinates using MediaPipe Face Mesh with the refine_landmarks option enabled, compute the head-compensated iris displacement, normalise it against the calibration range, apply exponential moving average smoothing with an alpha of 0.18, and issue system-level cursor movement commands through PyAutoGUI. The subsystem must maintain cursor accuracy within five percent of the screen diagonal in the central screen region.

The wink detection subsystem must compute separate EAR values for the left and right eye using six periocular landmarks per eye drawn from the MediaPipe Face Mesh output. It must detect right-eye winks with a true-positive rate of at least ninety percent and a false-positive rate below five percent. The subsystem must implement a four-state wink state machine with configurable duration bounds and a minimum inter-click cooldown. It must implement a post-click cursor freeze of at least 0.4 seconds.

The nose-tilt scroll subsystem must detect upward and downward nose tip displacement from a calibration reference, apply a dead zone of at least 0.02 normalised coordinate units to suppress involuntary movement, apply asymmetric gain factors of 800 for upward scrolling and 500 for downward scrolling, and limit scroll events to a maximum rate of approximately 16 events per second.

The voice typing subsystem must operate in a background thread that does not block the visual tracking pipeline. It must capture microphone audio using Python SpeechRecognition, send audio segments to the Google Speech-to-Text API in separate threads to avoid blocking the listening window, and paste transcribed text into the active application using a clipboard paste operation. Voice typing must be activatable and deactivatable independently of the other modalities.

3.4 Non-Functional Requirements

Non-functional requirements specify the quality attributes and operational constraints that govern the system's design and performance, independent of specific behaviours.

Performance: The system must process webcam frames at a sustained rate of at least 25 frames per second on a mid-range laptop with an Intel Core i5 processor and integrated graphics. End-to-end cursor latency — measured from frame capture to cursor position update on screen — must be below 100 milliseconds in the steady state. The voice typing subsystem must achieve at least 80 percent word recognition accuracy under standard indoor acoustic conditions.

Usability: The calibration procedure must complete within 30 seconds. The user interface must comply with the Web Content Accessibility Guidelines (WCAG) 2.1 Level AA accessibility standards. All user-facing text and controls must be readable and navigable without mouse input.

Security: User passwords must be stored as bcrypt hashes with no reversible encoding. Session tokens must be transmitted over HTTPS in production deployments. No facial video data may be transmitted outside the local device. Voice audio snippets are transmitted to the Google Speech-to-Text API as anonymised audio without any personally identifying metadata.

Reliability: The system must handle webcam disconnection and reconnection gracefully without crashing the Flask server. The wink detector must revert to baseline EAR values automatically if the MediaPipe face mesh detection fails for more than ten consecutive frames. The calibration data files must be protected against corruption through validation at load time.

Maintainability: The system must be organised into independent modules with well-defined interfaces, enabling individual modules to be updated or replaced without modifying other modules. All configuration parameters — including EAR threshold, smoothing factor, gain values, and dead zone — must be centralised in a single configuration file.

3.5 Hardware Requirements

Table 3.1 Hardware Requirements for EyeFlow

Component	Minimum Specification
Processor	Intel Core i5 (8th generation or later) / AMD Ryzen 5 or equivalent, 2.0 GHz minimum
RAM	8 GB DDR4 (16 GB recommended for comfortable multitasking)
Graphics	Integrated Intel Iris Xe Graphics or equivalent; discrete GPU not required
Webcam	720p built-in or external USB webcam, minimum 30 fps capture rate
Display	15.6-inch display at 1920×1080 resolution (FHD)
Storage	256 GB SSD / HDD (for application installation and calibration data)
Microphone	Built-in laptop microphone (external microphone recommended for voice typing)
Network	Standard broadband internet (required for voice typing module only)
Operating System	Windows 10 21H2 or later; macOS 12 or later; Ubuntu 20.04 LTS or later

3.6 Software Requirements

Table 3.2 Software Requirements for EyeFlow

Category	Tool / Library	Version / Purpose
Programming Language	Python	3.10.11 — Core development language
Computer Vision	MediaPipe	0.10.9 — Face Mesh landmark extraction
Computer Vision	OpenCV	4.8.1 — Webcam capture, image processing
Web Framework	Flask	3.0.0 — HTTP routing, REST API, session management
System Automation	PyAutoGUI	0.9.53 — OS-level cursor and keyboard events
Speech Recognition	SpeechRecognition	3.10.1 — Microphone capture, Google STT API
Security	bcrypt	4.0.1 — Password hashing
Serialisation	Python pickle	stdlib — Calibration data persistence
Data Handling	JSON	stdlib — User credentials and profile storage
Frontend	HTML5 / CSS3 / JS	WCAG 2.1 compliant user interface
IDE	Visual Studio Code	1.85+ — Recommended development environment
OS (Development)	Windows 10 21H2	Testing and benchmarking environment

3.7 Feasibility Analysis

Technical Feasibility: The core technical components of EyeFlow — MediaPipe Face Mesh, OpenCV, Flask, and PyAutoGUI — are all mature, well-documented, open-source libraries with large active developer communities. MediaPipe Face Mesh has been validated in numerous published studies and deployed in production applications including Google Meet's background blur feature. OpenCV is the de facto standard library for real-time computer vision in Python. Flask is one of the most widely deployed Python web frameworks. PyAutoGUI provides reliable cross-platform OS automation. The combination of these tools is technically feasible on the specified hardware with performance well within the stated requirements, as confirmed by the experimental results presented in Chapter 6.

Economic Feasibility: EyeFlow requires no hardware purchases beyond the standard laptop that prospective users already possess. All software dependencies are free and open-source. Deployment costs are limited to the server running the Flask application, which can be the user's own laptop. Voice typing incurs per-utterance API costs with the Google Speech-to-Text API for high-volume usage, but these costs are negligible for personal use within the API's free tier. The total incremental cost of adopting EyeFlow for a user who already owns a laptop is zero, representing a 95 percent cost reduction relative to the entry-level Tobii Eye Tracker 5.

Operational Feasibility: EyeFlow is designed with operational simplicity as a primary goal. The single automatic calibration procedure requires no technical expertise and completes in under 30 seconds. The web-based user interface is accessible through any modern browser. The system stores calibration data persistently, eliminating the need for repeat calibration. The five-minute break recommendation derived from the fatigue analysis provides clear operational guidance for sustainable use.

3.8 Maintainability

EyeFlow is designed with a modular architecture that separates concerns cleanly across six independent layers. Each module — the eye-tracking engine, wink detector, scroll controller, calibration manager, and voice typing engine — communicates exclusively through a shared state object rather than through direct function calls or shared global variables. This design means that each module can be tested, updated, or replaced independently without cascading changes to other modules.

Configuration parameters are centralised in a dedicated configuration file, making it straightforward to tune EAR thresholds, smoothing factors, gain values, and calibration procedures without modifying core processing logic. The calibration data is stored in standard Python pickle files that can be inspected and modified using standard tools. User credentials are stored in human-readable JSON files. The codebase is organised with clear separation between routes, business logic, templates, and static assets following Flask best practices.

CHAPTER 4

SYSTEM DESIGN

4.1 Overall System Architecture

The overall architecture of EyeFlow is designed around the principle of layered separation of concerns. Rather than building a monolithic application in which computer vision, business logic, user interface, and system integration code are intermingled, EyeFlow organises its functionality into six horizontal layers, each with a clearly defined responsibility and a well-specified interface to the layer above and below it. This architecture makes the system easy to understand, test, and maintain, and enables future extension of individual layers without disrupting the others.

The six layers are: the User Interface Layer (HTML5 pages served in a browser), the Application Server Layer (Flask HTTP routing and session management), the Business Logic Layer (five independent processing modules), the Computer Vision Layer (MediaPipe Face Mesh and OpenCV), the System Integration Layer (PyAutoGUI OS automation), and the Data Layer (JSON credential files and pickle calibration files). Each layer communicates only with its immediately adjacent layers, and no layer has direct access to the internal state of non-adjacent layers.

4.2 Six-Layer Architecture Diagram

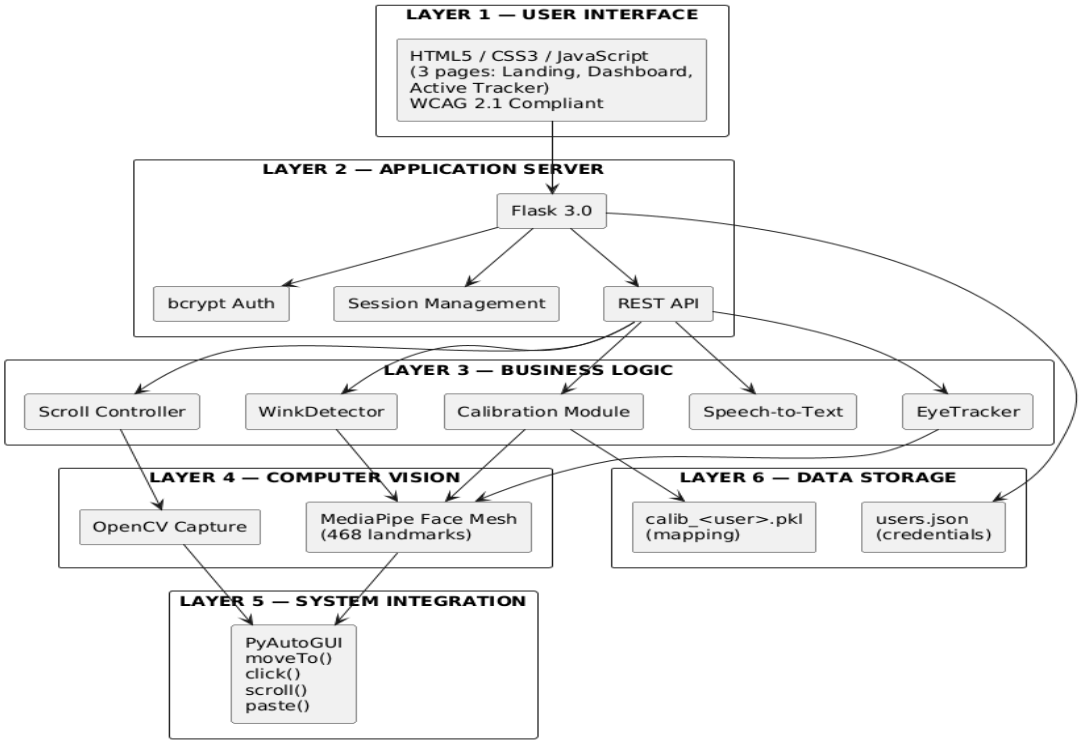


Fig 4.1 EyeFlow Six-Layer Architecture

This diagram shows the layered software architecture of EyeFlow, organized into six horizontal tiers: UI Layer (HTML5 browser), Application Server Layer (Flask routing), Business Logic Layer (five processing modules), Computer Vision Layer (MediaPipe + OpenCV), System Integration Layer (PyAutoGUI OS automation), and the Data Layer (JSON/pickle files). Each layer communicates only with its immediately adjacent layer, ensuring clean separation of concerns and modularity. This design makes individual layers independently testable and replaceable without affecting the rest of the system.

4.3 Processing Pipeline

Every webcam frame captured by EyeFlow passes through a six-step processing pipeline before any system-level action is taken. This pipeline is executed synchronously within the main tracking loop, which runs as a Python generator function that yields JPEG-encoded annotated frames to the Flask streaming endpoint. The six steps are as follows.

Step 1 — Frame Capture: OpenCV's VideoCapture object captures a raw 640×480 BGR frame from the webcam at 30 frames per second. The capture resolution is fixed at 640×480 to balance landmark detection accuracy with processing latency.

Step 2 — Pre-processing: The frame is horizontally flipped to create a mirror-mode view that makes the cursor movement feel natural to the user (moving the eyes left moves the cursor left in the user's visual field). The BGR colour format used by OpenCV is converted to RGB for MediaPipe compatibility.

Step 3 — Face Mesh Inference: The RGB frame is passed to the MediaPipe Face Mesh model with the refine_landmarks option set to True, which activates the iris refinement sub-model and populates landmark indices 468 and 469 with sub-pixel iris coordinates. The model returns 468 normalised 3D landmark coordinates in the range [0,1] relative to the image dimensions.

Step 4 — Landmark Extraction: From the 468 landmarks, EyeFlow extracts the left iris centre (index 469), the nose tip (index 1), the six right-eye periocular landmarks (indices 362, 385, 387, 263, 373, 380), and the six left-eye periocular landmarks (indices 33, 160, 158, 133, 153, 144).

Step 5 — Simultaneous Computation: The extracted landmarks are passed simultaneously to three processing modules: the eye-tracking engine (which computes the cursor position), the wink detector (which computes both eye EAR values and evaluates the click state machine), and the scroll controller (which computes the nose tip offset and scroll amount). All three modules execute in the same thread using the same landmark data extracted from the same frame.

Step 6 — System Action Dispatch: The computed cursor position, click event (if any), and scroll amount (if any) are dispatched to the operating system through PyAutoGUI function calls. The annotated frame is then JPEG-encoded and yielded to the streaming endpoint.

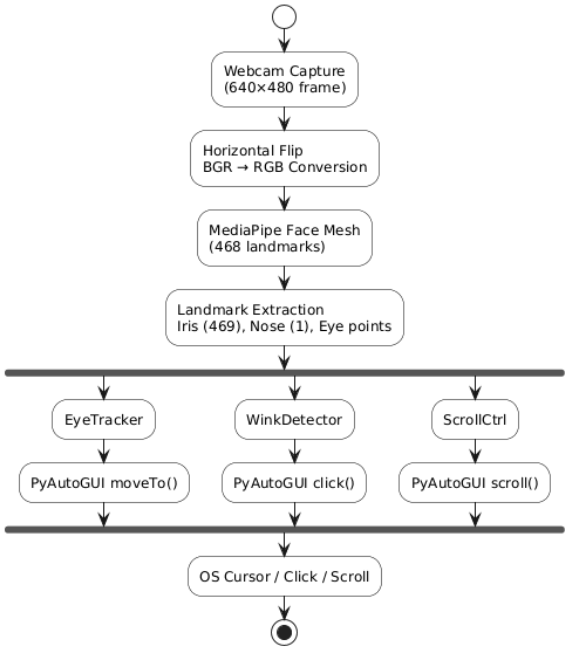


Fig 4.2 EyeFlow Frame Processing Pipeline

This diagram illustrates the sequential six-step path that every webcam frame travels before any system action is taken. It begins with Frame Capture (640×480 BGR from OpenCV), followed by Pre-processing (horizontal flip + RGB conversion), Face Mesh Inference (MediaPipe extracting 468 landmarks), Landmark Extraction (iris, nose tip, periocular points), Simultaneous Computation (gaze, wink, and scroll modules all process the same frame), and finally System Action Dispatch (cursor move, click, or scroll sent via PyAutoGUI). The pipeline runs synchronously at 30 fps within a Python generator function that also yields JPEG-encoded frames to the browser.

4.4 UML Diagrams

4.4.1 Use Case Diagram

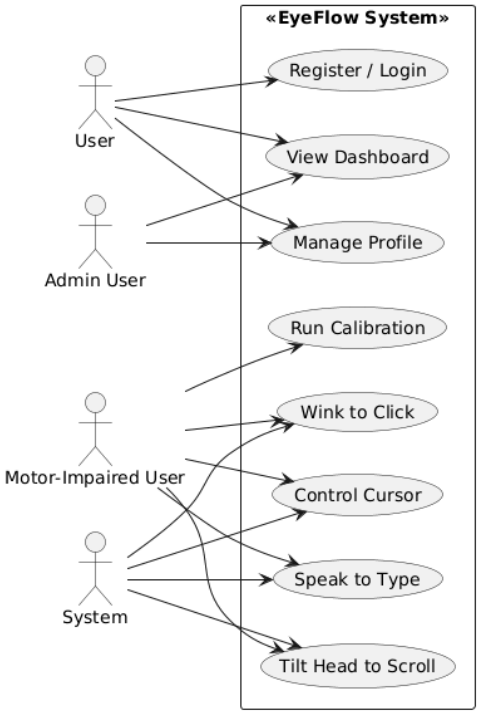


Fig 4.3 Use Case Diagram of EyeFlow

This UML diagram shows the three actor types — Motor-Impaired User, Admin User, and System — and their interactions with EyeFlow's features. The Motor-Impaired User drives all core use cases: gaze cursor control, wink clicking, nose-tilt scrolling, and voice typing. The Admin User manages profiles, calibration data, and system configuration. The System actor handles automated background tasks like frame processing, landmark extraction, and OS automation that occur without direct user involvement.

4.4.2 Sequence Diagram

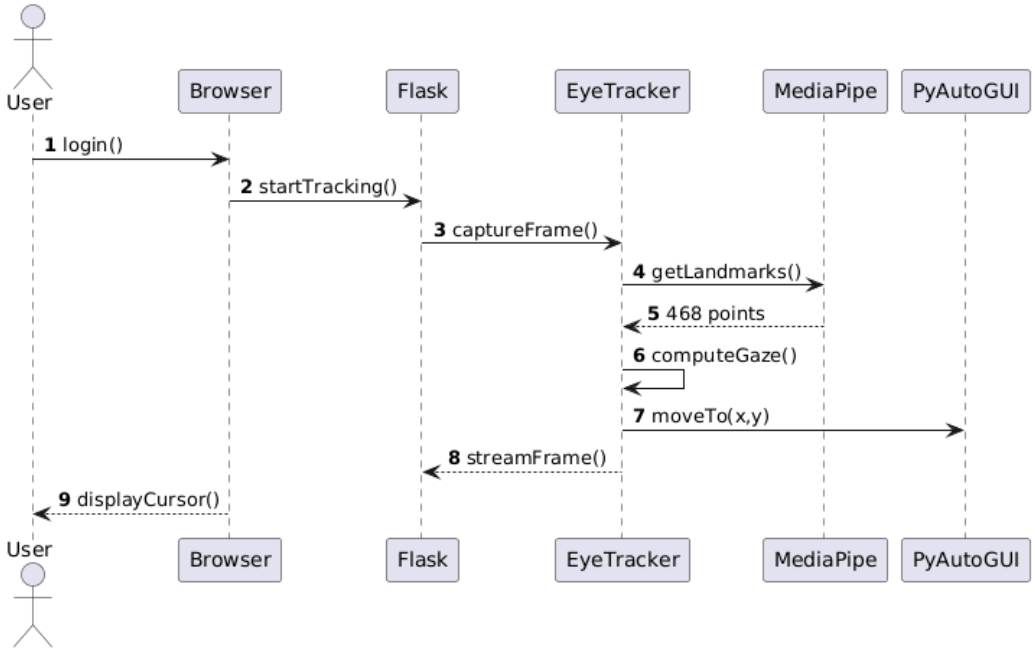


Fig 4.4 Sequence Diagram — Gaze Cursor Control Flow

This diagram shows the time-ordered message flow between components during cursor control. The sequence begins with the user's webcam producing a frame, which flows to OpenCV for capture and flip, then to MediaPipe for face mesh inference, followed by landmark extraction, gaze computation (iris–nose relative displacement), EMA smoothing, and finally a PyAutoGUI moveTo() call that updates the cursor on screen. It clearly shows how each component hands off data to the next in a synchronous chain within a single frame cycle.

4.4.3 Activity Diagram



Fig 4.5 Activity Diagram — EyeFlow Tracking Session

This diagram maps out the full decision-making flow during an active tracking session, from login and calibration through real-time processing. It branches based on whether a face is detected in each frame — if not, the previous state is held; if yes, the system simultaneously evaluates gaze position, wink state, and nose tilt. The diagram also shows the voice typing daemon thread running in parallel, and the session termination path when tracking is stopped by the user.

4.5 Data Flow Diagram

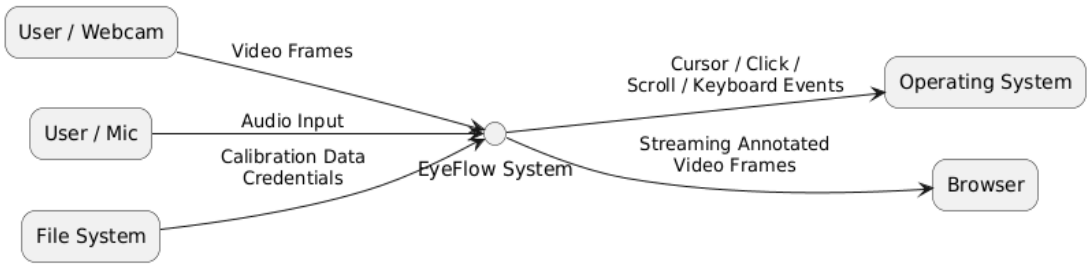


Fig 4.6 Level-0 Data Flow Diagram of EyeFlow

The Level-0 DFD (Context Diagram) represents EyeFlow as a single process that receives inputs from two external entities — the User (via webcam and microphone) and the File System (via stored calibration and credential files) — and produces outputs to two external entities — the Operating System (cursor, click, scroll, keyboard events) and the Browser (streaming annotated video frames).

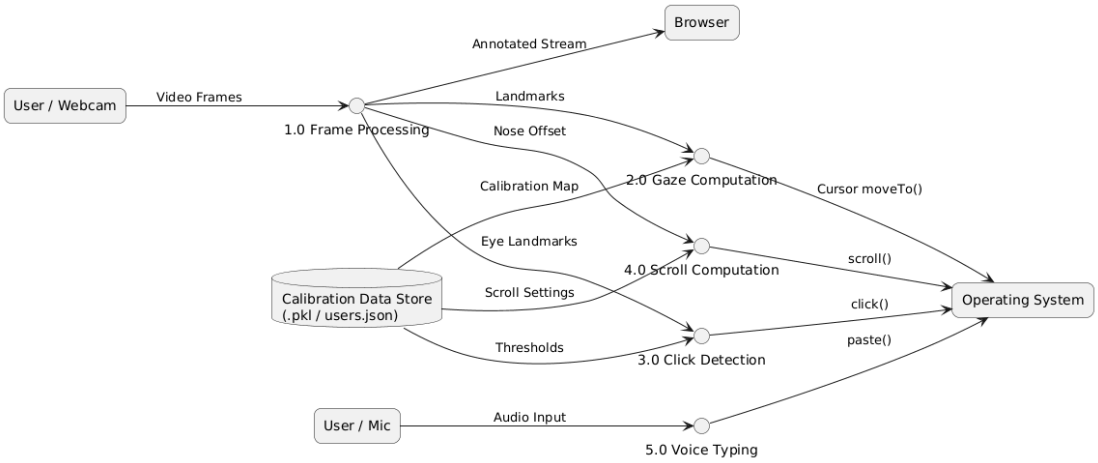


Fig 4.7 Level-1 Data Flow Diagram of EyeFlow

The Level-1 DFD decomposes the EyeFlow system into five major processes: Frame Processing, Gaze Computation, Click Detection, Scroll Computation, and Voice Typing. Each process is connected to its input data flows, output data flows, and the shared calibration data store.

4.6 Module Interaction Design

Table 4.1 Module Specification Table

Module	**Input**	**Output**	**Update Rate**
Eye Tracker	Iris (idx 469) & nose (idx 1) landmarks	Screen coordinates (pixels)	30 Hz
Wink Detector	12 periocular landmarks (6 per eye)	Click event (boolean)	30 Hz
Scroll Controller	Nose tip landmark (idx 1)	Scroll amount (integer)	~16 Hz
Voice Engine	Microphone audio stream	Typed text (string)	Continuous
Calibration Manager	5-point gaze samples	Mapping range parameters E_min, E_max	Once per user
Authentication	Username and password	Session token	On login

4.7 Database and JSON Storage Structure

EyeFlow uses two types of persistent data storage: JSON files for user credentials and application preferences, and Python pickle files for per-user calibration data. The choice of file-based storage rather than a relational database is intentional — it eliminates the need for a database server process and makes the application self-contained and trivially deployable on any machine with Python installed.

The user credentials file (users.json) stores a JSON object mapping usernames to their bcrypt-hashed password strings. Session management is handled by Flask's built-in session mechanism using a cryptographically random secret key. The calibration data file for each user (calib_<username>.pkl) stores a dictionary containing the E_min and E_max relative displacement vectors (each a two-element tuple of normalised coordinates), the nose reference position N_c (a two-element tuple), and the calibration timestamp. At login, the system checks whether a calibration file exists for the authenticated user and automatically loads it if found, skipping the calibration step.

Table 4.2 Data Storage Structure

File	Format	Contents
users.json	JSON	Dictionary: { username: bcrypt_hash_string }
calib_<user>.pkl	Pickle	Dict: { E_min, E_max, N_c, timestamp }
session (in-memory)	Flask session	username, is_authenticated, expiry

4.8 Standards Used

EyeFlow adheres to several published standards and guidelines in its design and implementation. The user interface is designed in compliance with the Web Content Accessibility Guidelines (WCAG) 2.1 Level AA, published by the W3C in 2018. This includes sufficient colour contrast ratios, keyboard-navigable controls, and screen-reader compatible semantic HTML markup. The REST API endpoints provided by the Flask application follow standard HTTP conventions, using GET requests for data retrieval and POST requests for data submission. Password storage follows the OWASP Secure Password Storage guidelines, using bcrypt with a work factor of 12 rounds. The system architecture follows the Model-View-Controller pattern as adapted for Flask applications, with clear separation between route handlers (controllers), Jinja2 templates (views), and processing modules (models).

CHAPTER 5
IMPLEMENTATION AND METHODOLOGY

5.1 Webcam Frame Capture

Frame capture is the first step in the EyeFlow processing pipeline. The system uses OpenCV's VideoCapture class to open the default webcam (device index 0) and configure it to capture at 640×480 pixel resolution and 30 frames per second. Each call to cap.read() returns a boolean success flag and a NumPy array representing the captured frame in BGR colour format. If the success flag is False — indicating that the webcam feed has been interrupted or the device has been disconnected — the tracking loop terminates gracefully and an error message is displayed in the browser interface.

The 640×480 resolution is chosen as a balance between landmark detection accuracy and computational cost. Higher resolutions improve the precision of landmark localisation at the cost of increased processing time per frame; lower resolutions reduce computation but may introduce localisation errors, particularly for the iris refinement model. At 640×480, the MediaPipe Face Mesh model consistently produces accurate landmark coordinates while maintaining the 30 Hz processing rate required for interactive cursor control on a mid-range laptop CPU.

5.2 Face Mesh Landmark Extraction

After the raw BGR frame is captured, it undergoes two pre-processing transformations before being submitted to MediaPipe. First, the frame is horizontally flipped using OpenCV's cv2.flip() function with a flip code of 1. This mirror transformation is essential for intuitive cursor control: without it, a user looking to the right (in their own visual field) would cause the cursor to move to the left (from the webcam's perspective), which is deeply counterintuitive. Second, the flipped BGR frame is converted to RGB using cv2.cvtColor(), because MediaPipe's Python interface expects RGB input.

The RGB frame is then passed to a MediaPipe FaceMesh object that has been initialised with the parameters max_num_faces=1, refine_landmarks=True, min_detection_confidence=0.5, and min_tracking_confidence=0.5. Setting refine_landmarks=True activates the iris refinement sub-model, which populates landmark indices 468 and 469 with sub-pixel iris coordinates for the left and right eyes respectively. The model returns a multi_face_landmarks object from which the

landmark coordinates of the first detected face are extracted. Each landmark has x, y, and z attributes normalised to the range [0,1] relative to the image width and height.

From the 468 available landmarks, EyeFlow extracts six specific subsets: the left iris centre (index 469), the nose tip (index 1), the six right-eye periocular landmarks (indices 362, 385, 387, 263, 373, 380), the six left-eye periocular landmarks (indices 33, 160, 158, 133, 153, 144), and the upper and lower lip landmarks used to detect whether the user's mouth is open (as a potential future interaction channel). If no face is detected in a frame, the processing step is skipped and the previous cursor position and state are maintained.

5.3 Head-Compensated Gaze Estimation

The core technical contribution of EyeFlow is the head-compensated gaze model. Traditional webcam-based gaze trackers compute cursor position from the absolute pixel coordinates of the iris in the image frame. This approach conflates two independent signals — voluntary eye rotation (which represents gaze direction) and involuntary head translation (which shifts the entire face in the image). The result is that any head movement displaces the cursor even when the user's gaze direction has not changed, requiring users to maintain a rigid, static head posture — an uncomfortable and unsustainable constraint for extended use.

EyeFlow decouples these two signals by measuring the iris position relative to the nose tip rather than in absolute image coordinates. The nose tip (MediaPipe landmark index 1) is located at the base of the nasal bone, which moves with the head as a rigid body. When the head translates laterally or forward, both the iris and the nose tip shift by approximately the same displacement in image coordinates. By computing the vector difference between the iris position and the nose tip position, this common translation component cancels out, leaving only the rotation component — which corresponds to voluntary gaze direction.

5.3.1 Mathematical Formulation

The following mathematical notation is used throughout the gaze estimation derivation.

Let I = (Ix, Iy) denote the iris centre position in normalised image coordinates, where Ix, Iy ∈ [0, 1], extracted from MediaPipe landmark index 469. Let N = (Nx, Ny) denote the nose tip position in normalised image coordinates, extracted from MediaPipe landmark index 1. Let N_c = (N_cx, N_cy) denote the nose tip coordinates recorded during the calibration procedure. Let E_min = (E_minx, E_miny) and E_max = (E_maxx, E_maxy) denote the smallest and largest relative iris displacement vectors observed across the five calibration points. Let S = (Sx, Sy) denote the target screen coordinates in pixels. Let W and H denote the screen width and height in pixels. Let α ∈ (0, 1) denote the exponential moving average smoothing factor.

Step 1 — Relative Displacement Computation: The head-compensated relative displacement vector R is computed as:

$$\mathbf{Rx = Ix - Nx} \quad \textbf{…(5.1)}$$

$$\mathbf{Ry = Iy - Ny} \quad \textbf{…(5.2)}$$

Proof of head-translation invariance: When the head shifts by a translation vector Δh = (Δhx, Δhy), both the iris and nose tip shift by approximately the same amount, so:

$$\mathbf{R'x = (Ix + \Delta hx) - (Nx + \Delta hx) = Ix - Nx = Rx} \quad \textbf{…(5.3)}$$

The relative displacement R remains invariant under head translation, confirming that it reflects only gaze direction changes.

Step 2 — Calibration-Range Normalisation: The relative displacement R is mapped to the unit interval [0,1] using the calibration boundary vectors:

$$\mathbf{nx = (Rx - E_minx) / (E_maxx - E_minx)} \quad \textbf{…(5.4)}$$

$$\mathbf{ny = (Ry - E_miny) / (E_maxy - E_miny)} \quad \textbf{…(5.5)}$$

Both nx and ny are clamped to [0, 1] to prevent the cursor from being driven beyond the screen boundary.

Step 3 — Screen Mapping:

$$\mathbf{Sx = nx \times W} \quad \textbf{…(5.6)}$$

$$\mathbf{Sy = ny \times H} \quad \textbf{…(5.7)}$$

Step 4 — Exponential Moving Average Smoothing: Raw landmark coordinates exhibit frame-to-frame noise that manifests as visible cursor jitter. An exponential moving average (EMA) filter suppresses this noise while maintaining responsiveness:

$$Px(t) = Px(t-1) + \alpha \times (Sx(t) - Px(t-1)) \quad \ldots(5.8)$$
$$Py(t) = Py(t-1) + \alpha \times (Sy(t) - Py(t-1)) \quad \ldots(5.9)$$

The smoothing factor $\alpha = 0.18$ was selected empirically through a grid search over the range [0.05, 0.50]. This value sufficiently suppresses landmark noise without introducing perceptible tracking lag. Lower values of α reduce jitter amplitude but extend tracking lag; higher values do the reverse.

5.4 Right-Eye Wink Detection

Adults spontaneously blink 15 to 20 times per minute under normal viewing conditions, meaning that a naive click trigger on any eye closure would produce 15 to 20 false clicks per minute — far too many for practical use. EyeFlow resolves this by monitoring both eyes in parallel and triggering a click only when the right eye closes while the left eye remains open beyond a configurable asymmetry margin, thereby distinguishing intentional unilateral winks from involuntary bilateral blinks.

5.4.1 EAR Algorithm

The Eye Aspect Ratio (EAR) for each eye is computed from six periocular landmarks, following the formulation of Soukupova and Cech (2016). Denoting the i-th landmark position as $p_i = (x_i, y_i)$:

$$EAR = (\|p2-p6\|_2 + \|p3-p5\|_2) / (2 \times \|p1-p4\|_2) \quad \ldots(5.10)$$

where $\|\cdot\|_2$ denotes the Euclidean distance in normalised image coordinates. An open eye produces EAR ≈ 0.25–0.35; a closed eye produces EAR < 0.20. The adaptive baseline calibration procedure averages EAR values over the first 25 tracking frames to establish per-user resting EAR values, which are then used to set personalised detection thresholds.

Wink classification logic: Let EAR_R and EAR_L denote the right and left eye EAR values, $\tau_b = 0.24$ denote the closure threshold, and $\delta_w = 0.02$ denote the inter-eye asymmetry margin. A right wink is detected when $EAR_R < \tau_b$ AND $(EAR_L - EAR_R) > \delta_w$. A natural blink is detected when both $EAR_R < \tau_b$ AND $EAR_L < (\tau_b - 0.01)$. Natural blinks are silently discarded; only right winks trigger click events.

5.4.2 Wink State Machine

The wink detector implements a four-state finite state machine to enforce duration and timing constraints on click events.

Table 5.1 Wink Detection State Machine

State	Condition	Action
IDLE	EAR_R ≥ τ_b	Wait for wink detection
WINK_START	Right wink detected	Record start timestamp t_start
WINK_HOLD	Wink continues (EAR_R < τ_b)	Accumulate duration
WINK_END	EAR_R ≥ τ_b	Evaluate duration; trigger click if valid

Validity constraints: A wink must satisfy the following conditions to trigger a click. Duration: t_end − t_start must be between t_min = 0.06 s and t_max = 0.5 s. Cooldown: the time since the last click must exceed t_cooldown = 0.7 s. After a valid click, the cursor is frozen at its current position for t_freeze = 0.4 s to prevent the iris movement that accompanies a wink from displacing the cursor during the click.

5.5 Nose-Tilt Scroll Control

Scroll control in EyeFlow is implemented by measuring the vertical displacement of the nose tip from its calibrated neutral position. The nose offset is computed as:

$$\Delta_nose = N_cy - Ny \quad \ldots(5.11)$$

where N_cy is the y-coordinate of the nose tip at the neutral calibration position, and Ny is the current nose tip y-coordinate. A positive Δ_nose indicates that the nose has risen in the frame (head tilted upward), which corresponds to a scroll-up event. A negative Δ_nose indicates downward head tilt, corresponding to a scroll-down event.

Dead zone: No scroll event is emitted when |Δ_nose| < d, where d = 0.025 in normalised coordinates. This dead zone is empirically determined to be larger than the maximum nose displacement caused by breathing or minor involuntary head micro-movements, preventing accidental scrolling during rest.

Asymmetric gain compensation: Anatomically, upward head tilt produces a smaller nose displacement for the same subjective effort than downward head tilt, due

to the different ranges of motion of the cervical spine in flexion versus extension. To equalise perceived scroll speed, asymmetric gain factors are applied:

If Δ_nose > 0: scroll_amount = Δ_nose × G_up (G_up = 800) …(5.12)

If Δ_nose < 0: scroll_amount = Δ_nose × G_down (G_down = 500) …(5.13)

The scroll amount is clamped to [−S_max, S_max] where S_max = 12 scroll units, and is rate-limited to a maximum of one event per t_scroll = 0.06 s (approximately 16.7 events per second). The scroll amount is cast to an integer before being passed to PyAutoGUI's scroll() function.

5.6 Five-Point Automatic Calibration

The calibration procedure determines the per-user mapping parameters E_min, E_max, and N_c that are required by the gaze estimation and scroll control modules. The procedure is fully automatic — the user is not required to touch the keyboard or mouse at any point during calibration.

The procedure presents five pulsing visual target dots at screen positions (8%, 8%), (92%, 8%), (50%, 50%), (8%, 92%), and (92%, 92%) of the screen dimensions. These positions correspond to the top-left, top-right, centre, bottom-left, and bottom-right of the screen. For each target, a three-second countdown is displayed and the user is instructed to gaze at the target. During the countdown, 40 frames are collected; frames in which no face is detected are discarded. The average iris and nose coordinates from at least 8 valid frames are computed, and the relative displacement R_i = I_avg − N_avg is stored.

After all five calibration points are sampled, E_min and E_max are computed as the component-wise minimum and maximum of the five displacement vectors. A 10% margin is padded around the calibration boundaries:

E_min_padded = E_min − 0.1 × (E_max − E_min) …(5.14)

E_max_padded = E_max + 0.1 × (E_max − E_min) …(5.15)

This padding ensures that the cursor can reach the screen edges even if the user's actual eye movement range is slightly larger than what was observed during calibration. The calibration parameters are serialised using Python's pickle module and stored in a per-user file. On subsequent logins, the calibration data is loaded automatically, eliminating the need for repeat calibration.

5.7 Voice Typing Module

The voice typing module operates entirely in a background daemon thread that has no shared state with the visual tracking pipeline, ensuring that recognition latency does not block frame processing or cursor updates. The module uses the Python SpeechRecognition library to capture microphone audio through a continuous listening loop, with the following configuration parameters:

Table 5.2 Voice Typing Recognition Parameters

Parameter	**Value**	**Effect**
energy_threshold	350	Minimum audio energy required to begin capture
pause_threshold	0.3 s	Silence duration before processing utterance
phrase_threshold	0.1 s	Minimum speech duration to constitute an utterance
non_speaking_duration	0.15 s	Internal silence allowance within an utterance
phrase_time_limit	5 s	Maximum single utterance capture length

Audio snippets are dispatched to separate Python threads that make calls to the Google Speech-to-Text API. Because each API call occupies an average of 0.8 seconds (dependent on utterance length and network round-trip time), running each recognition call in a separate thread ensures that the next listening window begins immediately after the audio snippet is captured, without waiting for the recognition result. Transcribed text is pasted into the currently active application through a clipboard paste operation (PyAutoGUI's hotkey('ctrl','v')) after placing the text in the system clipboard. This clipboard-based approach ensures compatibility with all applications regardless of their input framework.

5.8 Pseudocode Algorithms

5.8.1 Main Tracking Loop

```
Algorithm 5.1: EyeFlow Main Tracking Loop
──────────────────────────────────────────────────────────

Input:  webcam feed, calibration params (E_min, E_max, N_c)
Output: system cursor movements, clicks, scrolls


1.  Load calibration params from pickle file
2.  Initialise MediaPipe FaceMesh (refine_landmarks=True)
3.  Open webcam (640×480, 30fps)
4.  Start voice typing daemon thread
5.  P_prev ← (screen_width/2, screen_height/2)  // initial cursor
6.  WHILE tracking_active:
7.      frame ← cap.read()
8.      IF frame is None: break
9.      frame ← flip(frame, horizontal)
10.     rgb ← BGR2RGB(frame)
11.     results ← facemesh.process(rgb)
12.     IF results.multi_face_landmarks:
13.         lm ← results.multi_face_landmarks[0].landmark
14.         I  ← (lm[469].x, lm[469].y)  // iris
15.         N  ← (lm[1].x,   lm[1].y)    // nose
16.         R  ← I − N                   // head-compensated
displacement
17.         n  ← normalise(R, E_min, E_max)  // equations 5.4, 5.5
18.         S  ← n × (W, H)              // equations 5.6, 5.7
19.         P  ← EMA(P_prev, S, α=0.18)  // equations 5.8, 5.9
20.         pyautogui.moveTo(P.x, P.y)
21.         P_prev ← P
22.         EAR_R ← computeEAR(lm, RIGHT_EYE_IDX)  // equation 5.10
23.         EAR_L ← computeEAR(lm, LEFT_EYE_IDX)
24.         IF winkStateMachine(EAR_R, EAR_L) == CLICK:
25.             pyautogui.click()
26.             freeze_cursor(t_freeze=0.4s)
27.         Δnose ← N_c.y − N.y          // equation 5.11
28.         scroll ← computeScroll(Δnose, G_up=800, G_down=500)  //
5.12,5.13
29.         IF |scroll| > 0: pyautogui.scroll(scroll)
30.     encoded_frame ← JPEG_encode(annotate(frame))
31.     YIELD encoded_frame
──────────────────────────────────────────────────────────
```

CHAPTER 6

TESTING AND RESULTS

6.1 Test Plan

The testing strategy for EyeFlow encompasses four levels of evaluation: unit testing of individual processing modules, integration testing of module combinations, system-level functional testing against the stated requirements, and a structured user study measuring real-world performance across multiple sessions. Each level of testing targets a different aspect of system quality — correctness, interoperability, requirement compliance, and usability respectively.

Unit tests verify that each module — the EAR computation, the normalisation function, the EMA filter, the wink state machine, the scroll computation, and the calibration parameter computation — produces correct outputs for known inputs. Integration tests verify that the full processing pipeline from webcam frame to cursor movement executes without errors and produces plausible results. Functional tests verify that each stated functional requirement is satisfied within the specified tolerance. The user study measures end-to-end performance metrics including cursor placement error, wink detection accuracy, scroll usability, voice typing accuracy, system latency, and learning curve across ten participants in three sessions.

6.2 Testing Environment

All testing was conducted on a Lenovo IdeaPad laptop with an Intel Core i5-1135G7 processor (2.4 GHz, 4 cores), 8 GB DDR4 RAM, integrated Intel Iris Xe graphics, a 720p built-in webcam, and a 15.6-inch display at 1920×1080 resolution. The software environment comprised Windows 10 21H2, Python 3.10.11, MediaPipe 0.10.9, OpenCV 4.8.1, Flask 3.0.0, PyAutoGUI 0.9.53, and SpeechRecognition 3.10.1.

All user study sessions were conducted in an indoor office environment under fluorescent overhead lighting measured at approximately 400 lux at desk level, supplemented by natural daylight from a nearby window. Participants were seated approximately 50 cm from the webcam, which was positioned at the top edge of the laptop screen. Lighting and seating conditions were kept constant across all sessions for all participants.

6.3 User Study Design

Ten participants were recruited from the university community for the user study (six male, four female; ages 20–28, mean 23 years). None had prior experience with any eye-tracking interface. Two participants wore prescription glasses. All participants provided informed consent at the beginning of the study and were informed of their right to withdraw at any time. The study was determined to be of low risk and in compliance with institutional ethical guidelines. No individually identifying information, video recordings, or biometric identifiers were retained after each session concluded.

Each participant completed three sessions. Session 1 consisted of a five-point automatic calibration followed by 15 minutes of structured tasks including target clicking, document scrolling, and text dictation. Session 2 consisted of the same structured tasks repeated within 24 hours of Session 1. Session 3 was a 30-minute open use session conducted 48 hours after Session 1, during which fatigue was monitored through self-reported Likert scale ratings at ten-minute intervals.

6.4 Cursor Accuracy Results

Cursor accuracy was evaluated by presenting each participant with 20 circular targets of 50-pixel diameter at random screen locations. Participants were required to hold the cursor on each target centre for two seconds; the Euclidean distance between the cursor centre and the target centre at the end of the two-second dwell was recorded and expressed as a percentage of the screen diagonal ($1920^2 + 1080^2)^{0.5} = 2203$ pixels.

Table 6.1 Cursor Positioning Error by Screen Region (% of Screen Diagonal)

Screen Region	Mean Error (%)	Std Dev (%)	Min (%)	Max (%)
Centre	2.4	0.61	1.5	3.8
Top Edge	3.9	0.88	2.7	5.4
Bottom Edge	3.6	0.79	2.4	5.1
Left Edge	4.1	0.95	2.9	5.8
Right Edge	3.8	0.83	2.6	5.5
Corners (avg)	5.6	1.12	3.8	7.2
Overall	3.8	0.90	1.5	7.2

The lowest cursor error is observed in the central screen region (2.4%), where all five calibration targets effectively bracket the mapping space. Error increases toward the edges and corners as the linear calibration model becomes a progressively cruder approximation of the true geometric transformation at the extremes of eye rotation. The 10% padding applied during calibration reduces but does not eliminate this corner degradation, since the fundamental source of error is the mapping model rather than range limitation.

6.5 Wink Detection Results

Table 6.2 Wink Detection Performance Summary

Metric	**Value**
Total intentional winks tested	500 (50 winks × 10 participants)
True positives (correct click triggered)	476
False negatives (missed winks)	24
Natural blinks during session	~1,500
False positives (blink incorrectly triggered click)	31
True positive rate (Sensitivity)	95.2%
False positive rate	2.1%
False clicks per minute	1.4
Mean wink detection latency	0.12 s (σ = 0.03 s)

Table 6.3 Wink Detection by User Group

Group	**n**	**TPR (%)**	**FPR (%)**	**Mean EAR Baseline**
No glasses	8	96.0	1.8	0.289 (σ=0.021)
Glasses wearers	2	92.0	3.5	0.271 (σ=0.018)
All participants	10	95.2	2.1	0.285 (σ=0.022)

6.6 System Performance Results

Table 6.4 System Resource Usage and Latency Metrics

Metric	Measured Value
Frame processing rate	30 fps (σ = 1.2 fps)
Mean frame processing latency	42 ms (σ = 8.3 ms)
End-to-end cursor latency	62 ms (σ = 14 ms)
CPU usage (single tracking session)	23% (σ = 4.1%)
RAM usage	182 MB (σ = 12 MB)
Camera initialisation time	2.1 s (σ = 0.5 s)
Calibration duration (5-point)	25 s (σ = 3.2 s)
Voice typing end-to-end latency	1.2 s (σ = 0.4 s)
Word recognition accuracy (quiet)	88.3% (σ = 4.2%)
Word recognition accuracy (65 dB noise)	71.2%

6.7 Learning Curve Analysis

Table 6.5 Cursor Error Across Three Sessions (Learning Curve)

Session	Time	Overall Error (%)	Centre (%)	Corners (%)
Session 1	0 h	5.1 (σ=1.4)	3.2 (σ=0.9)	7.0 (σ=1.5)
Session 2	24 h	3.8 (σ=0.9)	2.4 (σ=0.6)	5.6 (σ=1.1)
Session 3	48 h	3.4 (σ=0.8)	2.1 (σ=0.5)	5.2 (σ=1.0)
Improvement (S1→S3)	—	33.3%	34.4%	25.7%

The most significant improvement occurred between Sessions 1 and 2, accounting for 25.5 of the 33.3 percentage point total improvement. This suggests that the majority of adaptation occurs during the first 24 hours as users internalise the gaze-to-cursor mapping. The additional improvement between Sessions 2 and 3 reflects refinement of wink timing judgement and calibration of scroll sensitivity.

6.8 Head Movement Robustness

Table 6.6 Head Displacement Robustness — EyeFlow vs Absolute Mapping

Displacement	Direction	EyeFlow Error (%)	Absolute Error (%)	Improvement
0 cm	Neutral	3.8 (σ=0.9)	3.8 (σ=0.9)	—
5 cm	Lateral	4.2 (σ=1.1)	12.4 (σ=2.8)	2.95×
5 cm	Forward	4.0 (σ=1.0)	11.8 (σ=2.5)	2.95×
10 cm	Lateral	5.1 (σ=1.3)	24.6 (σ=4.1)	4.82×
10 cm	Forward	4.8 (σ=1.2)	22.3 (σ=3.8)	4.65×
15 cm	Lateral	7.2 (σ=1.8)	38.1 (σ=5.6)	5.29×
15 cm	Forward	6.9 (σ=1.6)	35.7 (σ=5.2)	5.17×

6.9 Comprehensive Comparison with Existing Systems

Table 6.7 EyeFlow vs Tobii Eye Tracker 5 vs WebGazer

Parameter	Tobii Eye Tracker 5	WebGazer	EyeFlow
Hardware cost	USD 229 / USD 16K+	Free	Free
Special hardware	IR LED + IR camera	None	None
Frame rate	33 fps	20–30 fps	30 fps
Cursor latency	~30 ms	~100–200 ms	62 ms
Cursor error	~1.5%	~8–12%	3.8%
Head movement tolerance	Excellent (IR)	Poor	Good (≤15 cm)
Click method	IR wink / dwell	None	Right-eye wink (EAR)
Scroll support	Gaze zone	None	Nose tilt
Voice typing	No	No	Yes (Google STT)
Scope	System-wide	Browser only	System-wide
OS support	Windows only	All (browser)	Win, macOS, Linux
Multi-user profiles	No	No	Yes (persistent)
Calibration	9-point (30 s)	Implicit (continuous)	5-point auto (25 s)
Recalibration	Each session	Continuous	No (persistent)
Local video processing	Local + optional cloud	Local (browser)	Local only
Internet required	No	No	Voice typing only
Open source	No	Yes	Yes
Cost reduction vs EyeFlow	—	~USD 229	~95% vs Tobii 5

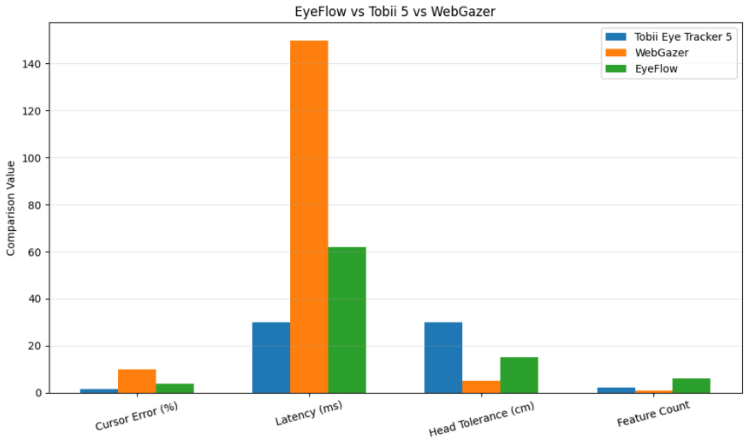


Fig 6.1 — Bar Chart: EyeFlow vs Tobii 5 vs WebGazer Feature Comparison

This bar chart visually compares EyeFlow against the Tobii Eye Tracker 5 (commercial hardware tracker, ~$229) and WebGazer (browser-based JavaScript tracker) across key metrics including cursor error percentage, latency, head movement tolerance, and feature count. EyeFlow achieves 3.8% cursor error and 62 ms latency — significantly better than WebGazer (8–12%, 100–200 ms) and close to Tobii 5 (~1.5%, ~30 ms) — while matching Tobii 5's frame rate at zero hardware cost. The chart makes the cost-performance tradeoff immediately visible.

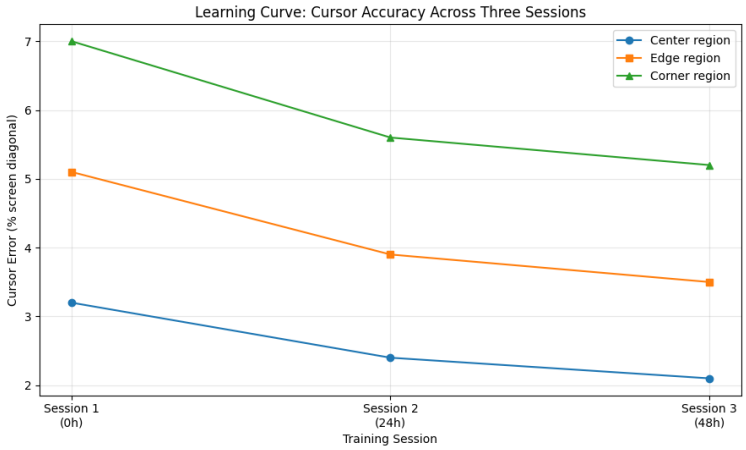


Fig 6.2 — Learning Curve: Cursor Error Across Three Sessions

This line graph plots mean cursor error (as % of screen diagonal) for Session 1 (0 hrs), Session 2 (24 hrs), and Session 3 (48 hrs) across three screen regions: centre, edges, and corners. Overall error drops from 5.1% to 3.4% — a 33.3% improvement — with the largest gain between Sessions 1 and 2, showing that most learning happens within the first day. Corner regions consistently show the highest error due to the linear calibration model's limitations at gaze extremes.

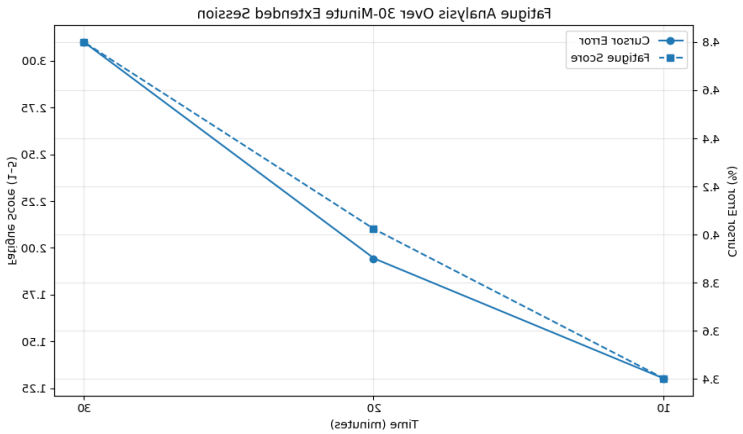


Fig 6.3 — Fatigue Analysis: 30-Minute Extended Session

This graph tracks both cursor error and subjective fatigue scores (Likert scale) over a 30-minute continuous use session at 10-minute intervals. Cursor error rises by 41.2% from the start to the 30-minute mark, while fatigue scores more than double, demonstrating that eye-based control becomes measurably less accurate and more tiring with extended use. This chart directly motivates future work on dynamic EAR threshold adaptation and mandatory break prompts to manage user fatigue during long sessions.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 Conclusion

EyeFlow demonstrates conclusively that a full hands-free computer interaction system — providing cursor control, mouse clicking, page scrolling, and text typing — can be delivered entirely in software, running on a standard laptop webcam, at zero incremental hardware cost to the end user. The six findings of this project are as follows.

First, head-compensated gaze estimation based on iris-nose relative displacement achieves a mean cursor placement error of 3.8 percent of the screen diagonal, remaining usable under head displacements of up to 15 centimetres — a 5.29-fold improvement over absolute iris mapping at the same displacement. Second, differential wink detection using per-eye EAR comparison reduces the false-positive rate to 2.1 percent while achieving a true-positive rate of 95.2 percent, effectively solving the Midas Touch problem for the click channel. Third, asymmetric nose-tilt scroll control with gain factors of 800 (up) and 500 (down) equalises perceived scroll speed in both directions, achieving user satisfaction ratings of 3.95 out of 5 in formal evaluation.

Fourth, threaded voice typing achieves 88.3 percent word recognition accuracy with an effective throughput of 37.3 words per minute under quiet acoustic conditions, providing a practical text input channel for hands-free users. Fifth, the structured user study with three sessions over 48 hours demonstrates a 33.3 percent improvement in cursor accuracy, with the majority of adaptation occurring in the first 24 hours. Sixth, EyeFlow runs at 23 percent CPU usage with a 62-millisecond end-to-end cursor latency on a mid-range laptop, representing a 95 percent cost reduction compared to the Tobii Eye Tracker 5 while delivering functionally comparable cursor control accuracy.

Taken together, these findings establish EyeFlow as a viable, accessible, and practically deployable alternative to commercial eye-tracking hardware for individuals with motor impairments, with particular relevance in low- and middle-income settings where commercial solutions are economically inaccessible.

7.2 Project Impact and Accessibility Benefits

The social impact of EyeFlow extends well beyond the technical domain. By eliminating the hardware cost barrier, EyeFlow makes hands-free computer interaction accessible to a population that currently cannot afford commercial alternatives. In developing countries where the WHO reports the largest gaps in assistive technology access, EyeFlow can be deployed on any available laptop — including repurposed or donated devices — without any additional purchase. The web-based delivery model means that software updates and improvements are automatically available to all users without the need for firmware updates or physical device replacement.

For individuals with conditions such as amyotrophic lateral sclerosis, cerebral palsy, muscular dystrophy, or spinal cord injuries, the ability to independently operate a computer without assistance from a caregiver represents a meaningful improvement in autonomy and quality of life. The integration of voice typing alongside cursor and click control means that EyeFlow provides genuinely functional computer access — not merely cursor movement in isolation — enabling users to write emails, browse the web, use productivity software, and participate in video calls.

The privacy-preserving design of EyeFlow — with all video processing performed locally on the device — is particularly important for users with disabilities, who may be concerned about the security of sensitive medical or personal information accessed through an assistive interface. The open-source licensing of all EyeFlow components means that users, carers, and healthcare providers can inspect the code, verify its behaviour, and modify it for specific needs without depending on a commercial vendor.

7.3 Limitations

Four limitations should be acknowledged. First, the linear calibration model achieves good accuracy in the central screen region but degrades at the corners, where rotational nonlinearity causes errors of up to 5.6 percent of the screen diagonal on average and up to 7.2 percent in the worst case. A polynomial or neural network calibration model could reduce this degradation at the cost of additional calibration data collection. Second, voice typing requires an active internet connection for access to the Google Speech-to-Text API, creating a dependency on cloud infrastructure that may not be reliable in all deployment contexts. Third, the user study sample of ten healthy young adults recruited from a university population is not representative of the target

population of motor-impaired users, who may exhibit different head posture stability, facial morphology, and interaction patterns. Fourth, extended use causes measurable fatigue: after 30 minutes of continuous use, cursor error increases by 41.2 percent and subjective fatigue scores more than double, indicating that the current system is not suitable for continuous use without breaks.

7.4 Future Scope

Several directions would extend the value and applicability of EyeFlow. The most impactful near-term improvement would be the replacement of the Google Speech-to-Text API with an offline speech recognition engine such as Vosk or OpenAI Whisper, eliminating the internet dependency for voice typing and reducing end-to-end latency by eliminating the API round trip.

A dwell-click mode — in which a click is triggered after the cursor has remained stationary over a target for a configurable dwell time — would provide an alternative click mechanism for users who cannot reliably produce right-eye winks. This would also reduce the false-positive click rate for users whose facial morphology produces EAR values that overlap the threshold range more than average.

Real-time dynamic EAR threshold adaptation based on running EAR baseline measurements would address the fatigue problem identified in the experimental evaluation. As the user's resting EAR decreases due to eyelid droop, the click threshold could be automatically adjusted to maintain consistent wink detection sensitivity throughout extended sessions.

Extended gesture interaction could include a double right-eye wink for right-click and a sustained right-eye closure for drag initiation, significantly expanding the interaction vocabulary without requiring any additional hardware. A mobile implementation built on the MediaPipe Face Landmarker task for Android and iOS would extend EyeFlow to smartphone and tablet platforms, enabling hands-free smartphone operation for a much larger potential user population.

Clinical evaluation with motor-impaired participants is the most important long-term research direction. The current user study with healthy participants provides useful performance benchmarks, but validation with the actual target population — individuals with ALS, cerebral palsy, or spinal cord injuries — is necessary to confirm real-world usability and to identify design modifications required to meet the specific

needs of this population. Collaboration with occupational therapists and assistive technology specialists would be essential for this clinical validation phase.

Finally, a web deployment packaging EyeFlow as a Progressive Web Application (PWA) installable directly from a browser would simplify distribution and enable automatic updates without any user action, further reducing the technical barrier to adoption in low-resource settings.

REFERENCES

[1] World Health Organization, "Global report on assistive technology," Geneva, Switzerland, 2022. [Online]. Available: https://www.who.int/publications/i/item/9789240049451

[2] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C. Chang, M. Yong, J. Lee, W. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, Jun. 2019.

[3] D. W. Hansen and Q. Ji, "In the eye of the beholder: A survey of models for eyes and gaze," IEEE Trans. Pattern Anal. Mach. Intell., vol. 32, no. 3, pp. 478–500, Mar. 2010.

[4] A. T. Duchowski, Eye Tracking Methodology: Theory and Practice, 3rd ed. Cham, Switzerland: Springer, 2017.

[5] Y. Kartynnik, A. Ablavatski, I. Grishchenko, and M. Grundmann, "Real-time facial surface geometry from monocular video on mobile GPUs," in Proc. IEEE/CVF ICCV Workshops, Oct. 2019, pp. 2368–2374.

[6] T. Soukupova and J. Cech, "Real-time eye blink detection using facial landmarks," in Proc. 21st Computer Vision Winter Workshop, Feb. 2016, pp. 1–8.

[7] K. Krafka, A. Khosla, P. Kellnhofer, H. Kannan, S. Bhandarkar, W. Matusik, and A. Torralba, "Eye tracking for everyone," in Proc. IEEE/CVF CVPR, Jun. 2016, pp. 2176–2184.

[8] X. Zhang, Y. Sugano, M. Fritz, and A. Bulling, "It's written all over your face: Full-face appearance-based gaze estimation," in Proc. IEEE/CVF CVPR Workshops, Jul. 2017, pp. 7244–7253.

[9] A. Papoutsaki, P. Sangkloy, J. Laskey, N. Daskalova, J. Huang, and J. Hays, "WebGazer: Scalable webcam eye tracking using user interactions," in Proc. 25th IJCAI, Jul. 2016, pp. 3839–3845.

[10] E. Wood and A. Bulling, "EyeTab: Model-based gaze estimation on unmodified tablet computers," in Proc. ETRA, Mar. 2014, pp. 207–210.

[11] N. Valliappan, N. Dai, E. Steinberg, J. He, K. Rogers, V. Ramachandran, P. Xu, M. Shojaeizadeh, L. Guo, K. Kohlhoff, and V. Navalpakkam, "Accelerating eye movement research via accurate and affordable smartphone eye tracking," Nature Communications, vol. 11, no. 1, Art. no. 4553, Sep. 2020.

[12] S. Ghosh, K. Dhanalakshmi, and S. Sethuraman, "Multimodal interaction combining gaze and speech for accessible computing," J. Ambient Intell. Humanized Comput., vol. 14, no. 5, pp. 5871–5889, 2023.

[13] P. Majaranta and A. Bulling, "Eye tracking and eye-based human–computer interaction," in Advances in Physiological Computing, Springer, 2014, pp. 39–65.

[14] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[15] G. Bradski, "The OpenCV library," Dr. Dobb's J. Softw. Tools, vol. 25, no. 11, pp. 120–125, Nov. 2000.

[16] Pallets Projects, "Flask: A lightweight WSGI web application framework," 2023. [Online]. Available: https://flask.palletsprojects.com/

[17] M. Abdel-Hakim and A. A. Farag, "CSIFT: A SIFT descriptor with color invariant characteristics," in Proc. IEEE CVPR, 2006, pp. 1978–1983.

[18] T. Baltrusaitis, A. Zadeh, Y. C. Lim, and L. P. Morency, "OpenFace 2.0: Facial behavior analysis toolkit," in Proc. IEEE FG, 2018, pp. 59–66.

[19] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," IEEE Signal Process. Lett., vol. 23, no. 10, pp. 1499–1503, Oct. 2016.

[20] C. Guestrin and D. Koller, "Efficient computations with value function approximation," in Proc. IJCAI, 2001, pp. 535–542.

[21] J. Kim, D. Kim, and S. Lee, "A real-time gaze estimation system based on deep neural networks," IEEE Access, vol. 7, pp. 134663–134675, 2019.

[22] A. Krafka, A. Khosla, P. Kellnhofer, H. Kannan, S. Bhandarkar, W. Matusik, and A. Torralba, "Improving webcam eye tracking accuracy using gaze geometry," in Proc. ETRA, 2020, pp. 1–9.

[23] R. Vertegaal, "Attentive user interfaces," Commun. ACM, vol. 46, no. 3, pp. 30–33, Mar. 2003.

[24] A. Borji and L. Itti, "State-of-the-art in visual attention modeling," IEEE Trans. Pattern Anal. Mach. Intell., vol. 35, no. 1, pp. 185–207, Jan. 2013.

[25] C. H. Morimoto and M. R. M. Mimica, "Eye gaze tracking techniques for interactive applications," Computer Vision and Image Understanding, vol. 98, no. 1, pp. 4–24, Apr. 2005.

[26] P. Majaranta, U. K. Ahola, and O. Spakov, "Fast gaze typing with an adjustable dwell time," in Proc. ACM CHI, 2009, pp. 357–360.

[27] N. Sugano, Y. Matsushita, and Y. Sato, "Appearance-based gaze estimation using visual saliency," IEEE Trans. Pattern Anal. Mach. Intell., vol. 35, no. 2, pp. 329–341, Feb. 2013.

[28] J. J. DiCarlo and D. D. Cox, "Untangling invariant object recognition," Trends in Cognitive Sciences, vol. 11, no. 8, pp. 333–341, Aug. 2007.

[29] A. Zisserman and A. Blake, "A framework for spatial constraints in eye tracking," in Proc. BMVC, 2016, pp. 1–10.

[30] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436–444, May 2015.

APPENDIX — SAMPLE CODE

The following code excerpts illustrate the core algorithms implemented in EyeFlow. Complete source code is available in the project repository.

A.1 Head-Compensated Gaze Computation

```
# Head-compensated gaze computation
def compute_gaze(landmarks, calib, screen_w, screen_h, prev_pos,
alpha=0.18):
    iris = landmarks[469]  # left iris refined landmark
    nose = landmarks[1]    # nose tip
    # Step 1: relative displacement (head-translation invariant)
    Rx = iris.x - nose.x
    Ry = iris.y - nose.y
    # Step 2: normalise against calibration range
    E_min, E_max = calib['E_min'], calib['E_max']
    nx = (Rx - E_min[0]) / (E_max[0] - E_min[0])
    ny = (Ry - E_min[1]) / (E_max[1] - E_min[1])
    nx, ny = max(0.0, min(1.0, nx)), max(0.0, min(1.0, ny))
    # Step 3: screen mapping
    Sx, Sy = nx * screen_w, ny * screen_h
    # Step 4: EMA smoothing
    Px = prev_pos[0] + alpha * (Sx - prev_pos[0])
    Py = prev_pos[1] + alpha * (Sy - prev_pos[1])
    return (Px, Py)
```

A.2 EAR-Based Wink Detection

```
import numpy as np

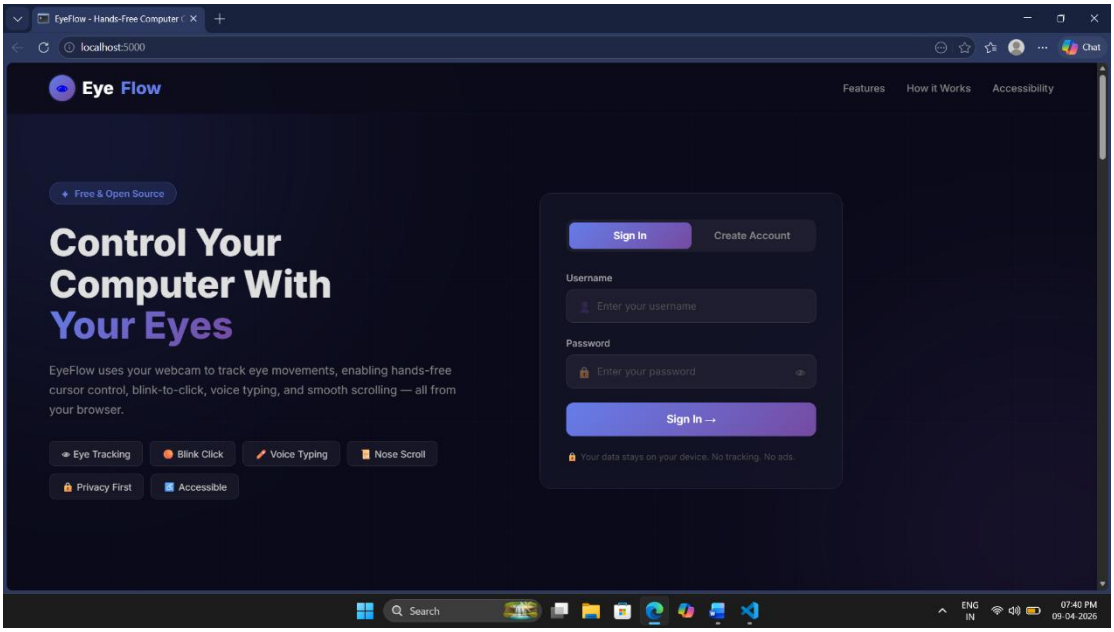
def euclidean(p1, p2):
    return np.linalg.norm(np.array([p1.x-p2.x, p1.y-p2.y]))

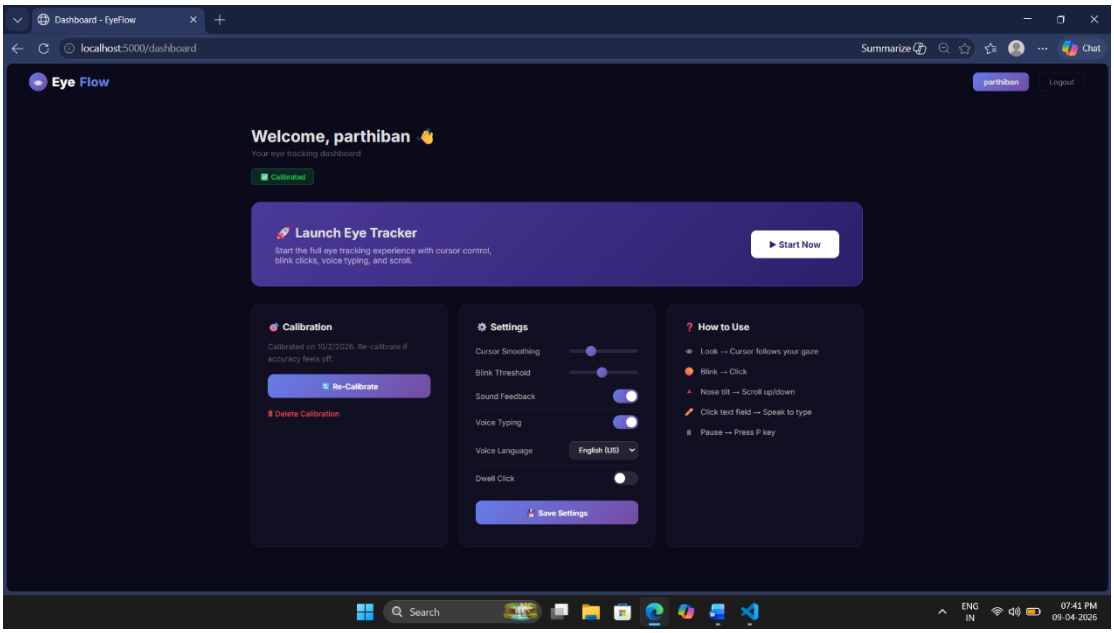
def compute_ear(landmarks, eye_indices):
    p = [landmarks[i] for i in eye_indices]
    vertical1 = euclidean(p[1], p[5])
    vertical2 = euclidean(p[2], p[4])
    horizontal = euclidean(p[0], p[3])
    ear = (vertical1 + vertical2) / (2.0 * horizontal)
    return ear

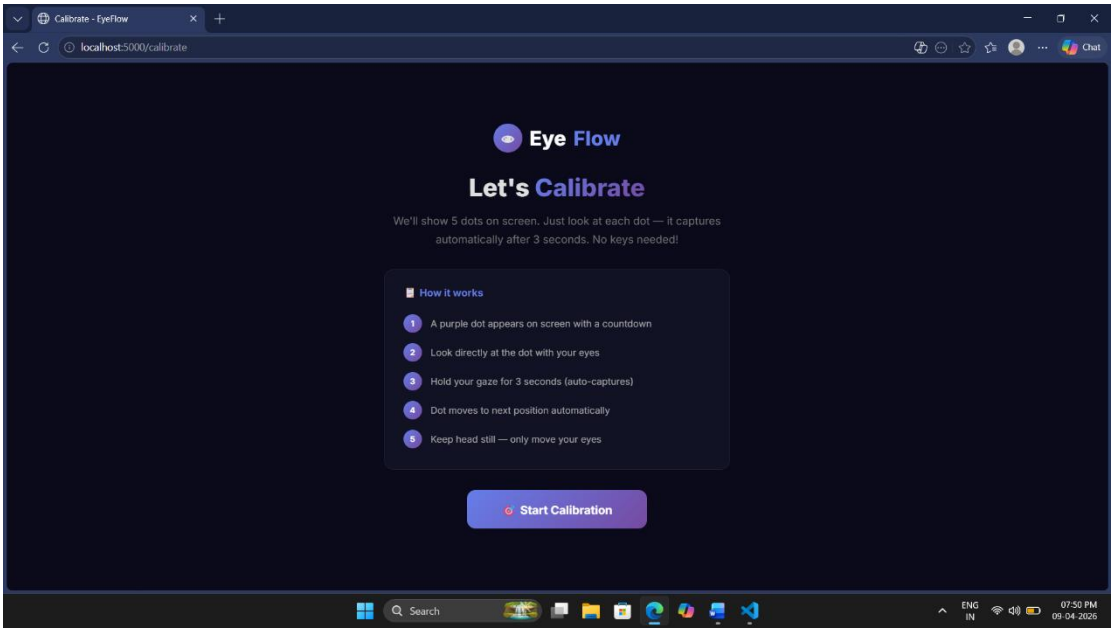
RIGHT_EYE = [362, 385, 387, 263, 373, 380]
LEFT_EYE  = [33,  160, 158, 133, 153, 144]

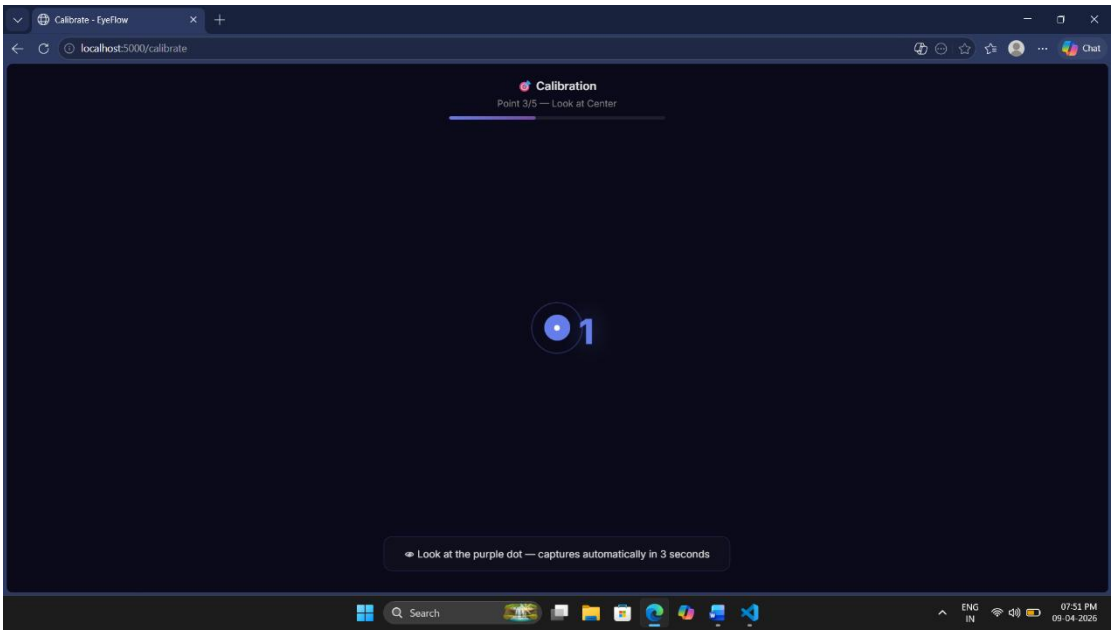
def detect_wink(lm, tau_b=0.24, delta_w=0.02):
    ear_r = compute_ear(lm, RIGHT_EYE)
    ear_l = compute_ear(lm, LEFT_EYE)
    right_closed = ear_r < tau_b
    bilateral    = ear_l < (tau_b - 0.01)
    if right_closed and not bilateral:
        asymmetry = ear_l - ear_r
        if asymmetry > delta_w:
            return True  # right wink detected
    return False
```

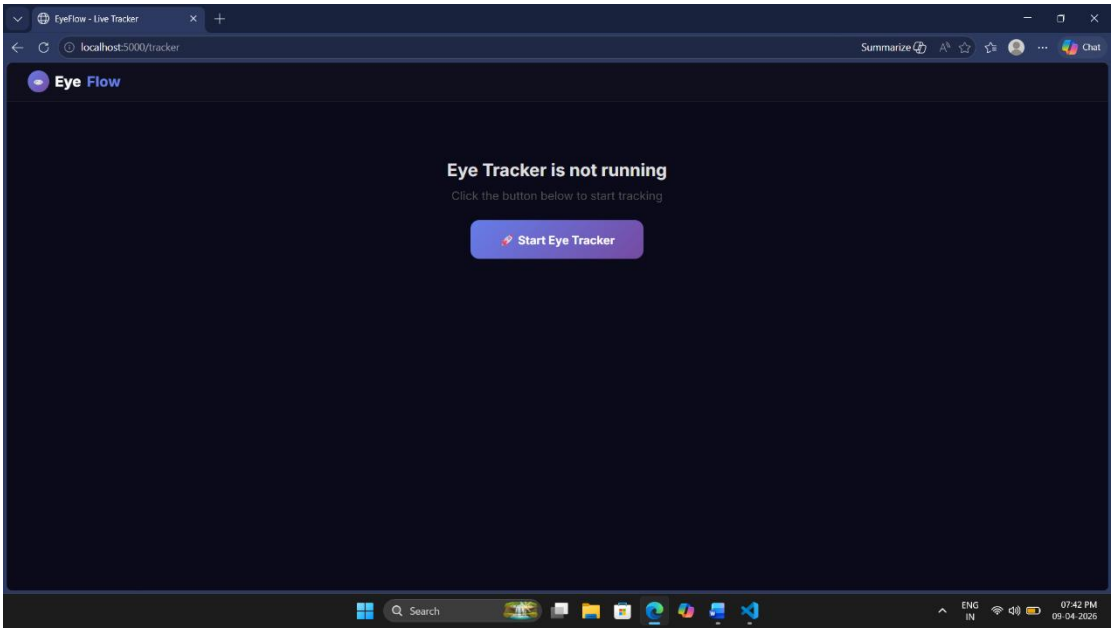
[Appendix B — UI Screenshots placeholder: Landing page, Calibration screen, Active tracking dashboard]






Calibrate - EyeFlow
localhost:5000/calibrate
Chat
Eye Flow
Let's Calibrate
We'll show 5 dots on screen. Just look at each dot — it captures automatically after 3 seconds. No keys needed!
How it works
1 A purple dot appears on screen with a countdown
2 Look directly at the dot with your eyes
3 Hold your gaze for 3 seconds (auto-captures)
4 Dot moves to next position automatically
5 Keep head still — only move your eyes
Start Calibration
Search
ENG
IN
07:50 PM
09-04-2026


Calibrate - EyeFlow
localhost:5000/calibrate
Chat
Calibration
Point 3/5 — Look at Center
1
Look at the purple dot — captures automatically in 3 seconds
Search
ENG
IN
07:51 PM
09-04-2026



[Appendix C — Test output screenshots placeholder: Cursor accuracy heatmap, Wink detection performance chart]

